

操作系统课程设计

xv6 实验报告

题目：基于 MIT xv6-labs-2021 的 RISC-V xv6 实验实现

实验范围：util / syscall / pgtbl / traps / cow / thread / net / lock / fs / mmap

姓名：_____ 学号：_____ 班级：_____

日期：2026-07-22

独立完成说明

- 本报告依据课程设计任务书、官方实验说明、本项目源码和真实测试日志撰写。
- 报告内容按实验流程展开，重点说明实现原理、关键修改、测试方法和问题处理。

目录

目录.....	2
摘要.....	10
课程设计任务与完成范围.....	10
实验环境与材料目录.....	10
分支与代码位置总览.....	11
实验零 环境配置与版本核查.....	12
实验目的.....	12
环境对照.....	12
配置步骤.....	13
本项目动态测试命令.....	13
遇到的问题和解决方法.....	13
环境配置心得.....	14
实验一 Xv6 and Unix utilities.....	15
实验目的.....	15
实验内容.....	15
任务分解与验收标准.....	15
实验原理与总体设计.....	17
关键数据结构与接口.....	17
源码阅读与修改依据.....	18
实现步骤.....	19
Boot xv6 (easy): 启动与退出系统.....	19
建立用户工具程序.....	19
实现 sleep	19
实现 pingpong	20
实现 primes.....	20
实现 find	21
实现 xargs.....	21
实现流程检查清单.....	21
边界条件清单.....	22

关键代码摘录.....	23
易错点与处理.....	23
实验中遇到的问题和解决方法.....	23
测试方案与结果记录.....	24
测试记录与核验说明.....	25
实验小结.....	26
实验二 System calls	27
实验目的.....	27
实验内容.....	27
任务分解与验收标准.....	27
实验原理与总体设计.....	29
关键数据结构与接口.....	29
源码阅读与修改依据.....	30
实现步骤.....	31
梳理系统调用入口.....	31
实现 trace 系统调用	31
实现 sysinfo	31
处理并发与一致性.....	32
实现流程检查清单.....	32
边界条件清单.....	33
关键代码摘录.....	34
易错点与处理.....	34
实验中遇到的问题和解决方法.....	34
测试方案与结果记录.....	35
测试记录与核验说明.....	36
实验小结.....	37
实验三 Page Tables	38
实验目的.....	38
实验内容.....	38
任务分解与验收标准.....	38

实验原理与总体设计.....	40
关键数据结构与接口.....	40
源码阅读与修改依据.....	40
实现步骤.....	42
实现 USYSCALL 映射.....	42
打印页表结构.....	42
实现 pgaccess	42
保持页表维护对称.....	43
实现流程检查清单.....	43
边界条件清单.....	43
关键代码摘录.....	45
易错点与处理.....	45
实验中遇到的问题和解决方法.....	45
测试方案与结果记录.....	46
测试记录与核验说明.....	47
实验小结.....	48
实验四 Traps	49
实验目的.....	49
实验内容.....	49
任务分解与验收标准.....	49
实验原理与总体设计.....	51
关键数据结构与接口.....	51
源码阅读与修改依据.....	52
实现步骤.....	53
RISC-V 调用约定分析	53
实现 backtrace	53
实现 sigalarm.....	53
实现 sigreturn	54
实现流程检查清单.....	54
边界条件清单.....	55

关键代码摘录.....	56
易错点与处理.....	56
实验中遇到的问题和解决方法.....	56
测试方案与结果记录.....	57
测试记录与核验说明.....	58
实验小结.....	59
实验五 Copy-on-write Fork	60
实验目的.....	60
实验内容.....	60
任务分解与验收标准.....	60
实验原理与总体设计.....	62
关键数据结构与接口.....	62
源码阅读与修改依据.....	62
实现步骤.....	64
重新设计 fork 的页面复制.....	64
维护物理页引用计数.....	64
处理写缺页.....	64
补齐 copyout 路径.....	65
实现流程检查清单.....	65
边界条件清单.....	65
关键代码摘录.....	67
易错点与处理.....	67
实验中遇到的问题和解决方法.....	67
测试方案与结果记录.....	68
测试记录与核验说明.....	69
实验小结.....	70
实验六 Multithreading.....	71
实验目的.....	71
实验内容.....	71
任务分解与验收标准.....	71

实验原理与总体设计.....	73
关键数据结构与接口.....	73
源码阅读与修改依据.....	73
实现步骤.....	75
实现用户级线程上下文.....	75
编写汇编切换例程.....	75
优化哈希表并发访问.....	75
实现 barrier.....	76
实现流程检查清单.....	76
边界条件清单.....	77
关键代码摘录.....	78
易错点与处理.....	78
实验中遇到的问题和解决方法.....	78
测试方案与结果记录.....	79
测试记录与核验说明.....	80
实验小结.....	81
实验七 Network Driver	82
实验目的.....	82
实验内容.....	82
任务分解与验收标准.....	82
实验原理与总体设计.....	84
关键数据结构与接口.....	84
源码阅读与修改依据.....	84
实现步骤.....	86
理解描述符环.....	86
实现发送路径.....	86
实现接收路径.....	86
处理错误与资源边界.....	87
实现流程检查清单.....	87
边界条件清单.....	88

关键代码摘录.....	89
易错点与处理.....	89
实验中遇到的问题和解决方法.....	89
测试方案与结果记录.....	90
测试记录与核验说明.....	91
实验小结.....	92
实验八 Locks.....	93
实验目的.....	93
实验内容.....	93
任务分解与验收标准.....	93
实验原理与总体设计.....	95
关键数据结构与接口.....	95
源码阅读与修改依据.....	95
实现步骤.....	97
分析 kmem 锁竞争.....	97
实现 per-CPU allocator	97
分析 bcache 锁竞争	97
处理 bcache miss 与回收	98
实现流程检查清单.....	98
边界条件清单.....	99
关键代码摘录.....	100
易错点与处理.....	100
实验中遇到的问题和解决方法.....	100
测试方案与结果记录.....	101
测试记录与核验说明.....	102
实验小结.....	102
实验九 File System.....	104
实验目的.....	104
实验内容.....	104
任务分解与验收标准.....	104

实验原理与总体设计.....	106
关键数据结构与接口.....	106
源码阅读与修改依据.....	106
实现步骤.....	108
扩展 inode 块映射.....	108
修改 bmap.....	108
修改 itrunc	108
实现 symlink.....	109
实现流程检查清单.....	109
边界条件清单.....	109
关键代码摘录.....	111
易错点与处理.....	111
实验中遇到的问题和解决方法.....	111
测试方案与结果记录.....	112
测试记录与核验说明.....	113
实验小结.....	114
实验十 Mmap	115
实验目的.....	115
实验内容.....	115
任务分解与验收标准.....	115
实验原理与总体设计.....	117
关键数据结构与接口.....	117
源码阅读与修改依据.....	118
实现步骤.....	119
设计 VMA 数据结构	119
实现 mmap 系统调用.....	119
处理缺页装入.....	119
实现 munmap 与写回.....	120
处理 fork、exit 和 exec.....	120
实现流程检查清单.....	121

边界条件清单.....121

关键代码摘录.....122

易错点与处理.....122

实验中遇到的问题和解决方法.....122

测试方案与结果记录.....123

 测试记录与核验说明.....124

实验小结.....125

综合验证与复现说明.....126

共性问题与调试分析.....126

 生命周期对称.....126

 权限位与状态位.....126

 并发语义.....126

 测试真实性.....126

项目成果说明.....127

总结.....127

附录 A：patch 文件清单.....127

附录 B：参考资料.....127

说明：本目录为 Word 标准 TOC 域，已设置打开文档时更新域。

摘要

本课程设计依据《xv6 及 Labs 课程项目（2026）》要求，选择 MIT 6.S081 / xv6-labs-2021 作为主线，完成 util、syscall、pgtbl、traps、cow、thread、net、lock、fs、mmap 共十个实验。报告重点记录实验目的、实验内容、原理设计、关键代码、实现步骤、测试方案、问题分析和结果记录。

在实验过程中，代码实现、分支管理、补丁文件、测试日志和运行截图均按实验顺序整理。每个实验都先分析官方要求和 xv6 相关源码，再完成代码修改，最后通过 make grade 或对应子测试进行验证。

课程设计任务与完成范围

- 课程任务书要求先阅读 Tools 和 Guidance，再完成新版 RISC-V xv6 实验，并在报告中注明相关代码、实验结果及分析。
- 报告不应只堆运行截图，也不应只列代码；应说明实验原理、实现路径、关键修改和测试依据。
- 报告按照课程设计报告要求组织目录层级、环境说明、分章步骤、问题处理、测试记录和总结。
- 测试截图与 real_test_logs 中的原始 make grade 输出对应；net 测试额外记录了网络绑定和 QEMU E1000 相关兼容处理。

实验环境与材料目录

项目	内容
实现仓库	work/xv6-labs-2021-solution
材料目录	outputs/xv6-course-project
代码分支	solution-util, solution-syscall, solution-pgtbl, solution-traps, solution-cow, solution-thread, solution-net, solution-lock, solution-fs, solution-mmap
可复现材料	xv6-labs-2021-solution.bundle, patches/*.patch, branch_commits.csv, runbook.md
推荐运行环境	Ubuntu / WSL Ubuntu / Docker，安装 git、build-essential、gdb-multiarch、qemu-system-misc、ipxe-qemu、gcc-riscv64-unknown-elf

分支与代码位置总览

Lab	分支	Commit	相关代码位置	实现摘要
util	solution-util	0220fd0	user/sleep.c; user/pingpong.c; user/primes.c; user/find.c; user/xargs.c; Makefile	完成用户态工具程序，覆盖进程、管道、递归文件遍历和参数批处理。
syscall	solution-syscall	13a2f9e	kernel/sysproc.c; kernel/syscall.c; kernel/proc.c; kernel/kalloc.c; user/trace.c	新增 trace 和 sysinfo 系统调用，补齐用户态 stub、内核分派、进程继承和信息拷贝。
pgtbl	solution-pgtbl	d27dab0	kernel/proc.c; kernel/vm.c; kernel/sysproc.c; kernel/riscv.h	实现 USYSCALL/ugetpid、vmprint 和 pgaccess，练习 Sv39 页表遍历与 PTE 标志维护。
traps	solution-traps	7e3990f	kernel/trap.c; kernel/proc.h; kernel/sysproc.c; kernel/riscv.h	实现 backtrace、sigalarm、sigreturn，并保存/恢复用户陷入现场。
cow	solution-cow	e09d1c9	kernel/vm.c; kernel/kalloc.c; kernel/trap.c; kernel/riscv.h	实现 copy-on-write fork、物理页引用计数、写缺页复制和 copyout COW 处理。
thread	solution-thread	557ce66	user/uthread.c; user/uthread_switch.S; notxv6/ph.c; notxv6/barrier.c	实现用户级线程切换、哈希表桶锁优化和 pthread barrier。
net	solution-net	33ee4c4	kernel/e1000.c; kernel/net.c; kernel/pci.c	实现 E1000 描述符环的发送与接收，处理 mbuf 生命周期和锁边界。
lock	solution-lock	5ebad33	kernel/kalloc.c; kernel/bio.c	将物理页分配器拆成 per-CPU freelist，将 bcache 改为哈希桶锁以降低竞争。
fs	solution-fs	7c6dae2	kernel/fs.h; kernel/file.h; kernel/fs.c; kernel/sysfile.c; user/usys.pl	实现大文件双重间接块、symlink 系统调用、O_NOFOLLOW 和 open 链接解析。
mmap	solution-mmap	ea8b72c	kernel/proc.h; kernel/sysfile.c; kernel/trap.c; kernel/proc.c; kernel/exec.c	实现 mmap/munmap、VMA 表、按需文件页装入、MAP_SHARED 写回和生命周期清理。

实验零 环境配置与版本核查

本章说明

- 本章说明 xv6-labs-2021 的编译、运行、调试和评分环境。
- 本项目依赖 Linux、QEMU 和 RISC-V 工具链，并采用 Docker/Ubuntu 完成真实动态测试。
- 实验范围严格按 xv6-labs-2021 路线组织，不额外并入其他年份的独立实验。

实验目的

- 建立可复现的 xv6-labs-2021 编译、运行、调试和评分环境。
- 确认 QEMU、RISC-V 交叉编译器、GNU Make、Git 和网络实验依赖均可用。
- 记录本次实验的测试日志、截图和相关材料，使实验结果能够被复核。
- 区分 WSL/Ubuntu、Docker/Ubuntu 和普通 Linux 虚拟机路线的共同点与差异。

环境对照

项目	本项目实际配置	说明
实验主机	Windows + Ubuntu 工具链环境	用于编辑源码、运行 Ubuntu 环境并保存测试日志。
Linux 环境	Ubuntu 容器环境	执行 make、QEMU、RISC-V GCC、grade 脚本，保证测试在类 Linux 环境中完成。
核心工具链	qemu-system-riscv64、riscv64-unknown-elf-gcc、GNU Make	满足 xv6-labs-2021 对 RISC-V 模拟器、交叉编译器和构建工具的要求。
补充依赖	gdb-multiarch、ipxe-qemu、git、build-essential	支持调试、网络实验 E1000 ROM、版本管理和基本编译。
测试产物	real_test_logs、browser_screenshots、xv6.out、packets.pcap	保留完整 make grade 证据链，网络实验额外保存包捕获相关产物。

本次动态测试采用 Ubuntu 工具链完成；复现实验时也可使用 WSL Ubuntu、Linux 虚拟机或 Docker Ubuntu 容器。三种路线的本质相同：都在 Linux 用户空间中使用 QEMU 模拟 RISC-V xv6，并用交叉编译器生成内核和用户程序。

配置步骤

1. 安装或准备 Linux 环境：可使用 WSL Ubuntu、Linux 虚拟机，或 Docker Desktop 中的 Ubuntu 容器。
2. 安装基础依赖：git、build-essential、GNU Make、QEMU RISC-V、GDB 和 RISC-V GCC。
3. 克隆或恢复 xv6-labs-2021 仓库，分别切换到 util、syscall、pgtbl、traps、cow、thread、net、lock、fs、mmap 分支。
4. 先运行 `make qemu` 验证系统能启动，再逐实验运行 `make grade` 验证实现正确性。

```
sudo apt-get update
sudo apt-get install -y git build-essential gdb-multiarch qemu-system-misc ipxe-qemu gcc-riscv64-unknown-elf
qemu-system-riscv64 --version
riscv64-unknown-elf-gcc --version
make --version
```

本项目动态测试命令

本报告对应的动态测试均在 Ubuntu 工具链中执行，测试过程按实验分支分别运行 `make grade`，并把原始输出保存到 `real_test_logs` 目录。

```
git checkout solution-util
make clean
make grade

# 逐实验复核时可按相同方式切换分支：
for lab in util syscall pgtbl traps cow thread net lock fs mmap; do
    git checkout solution-$lab
    make clean
    make grade
done
```

遇到的问题解决方法

问题	原因分析	解决方法
新版 GCC 把 <code>user/sh.c</code> 的递归警告视作错误	部分 Ubuntu/GCC 13 环境会出现 <code>-Werror=infinite-recursion</code> 。	在 <code>grading</code> 运行中加入 <code>-Wno-infinite-recursion</code> 作为工具链兼容参数，不改变 xv6 内核/用户程序语义。
Docker 网络实验中 <code>server.py</code> 仅绑定 <code>localhost</code>	容器网络与 QEMU user networking 的可达性和普通 WSL/物理机不同。	容器内测试时将 <code>server.py</code> 绑定到 <code>0.0.0.0</code> ，使 QEMU 网络测试可访问宿主服务。
QEMU 找不到 <code>efi-e1000.rom</code>	Ubuntu 包把 E1000 ROM 放在 <code>ipxe-qemu</code> 路径下，QEMU 默认查找路径不一定一致。	安装 <code>ipxe-qemu</code> ，并建立 <code>/usr/share/qemu/efi-e1000.rom</code> 到实际 ROM 文件的软链接。
测试截图与原始日志需要一一对应	若只保留终端示意图，无法充分说明测试确实执行。	从 <code>real_test_logs</code> 整理测试截图，并保留原始日志供复核。

环境配置心得

环境配置不是附属工作，而是操作系统实验的一部分。xv6 实验同时依赖编译器、模拟器、脚本、文件系统镜像和网络模拟，任何一个环节不一致都可能导致测试结果不同。本次实验保留 Ubuntu 工具链版本、测试命令和原始日志，是为了使最终结果不仅可见，而且可复现。

复现实验时，交互式调试适合在 WSL Ubuntu 中完成，最终评分测试适合在干净的 Ubuntu 工具链环境中执行。这样既能保持调试效率，也能降低本机环境差异对测试结果的影响。

实验一 Xv6 and Unix utilities

本章索引

- 分支: solution-util; 提交号: 0220fd0
- patch: outputs/xv6-course-project/patches/util.patch
- 本章包含目的、内容、原理图、文件职责、实现步骤、代码摘录、测试记录和小结。

实验目的

- 熟悉 xv6 用户态程序的组织方式，理解 user 目录、Makefile 中 UPROGS 目标和系统调用头文件之间的关系。
- 练习 fork、exec、wait、pipe、read、write、close、sleep 等最基础的 Unix 接口，建立“用户程序通过系统调用进入内核”的直观认识。
- 通过 primes 的管道筛法理解进程之间只共享文件描述符和管道数据流，而不是共享普通用户空间变量。
- 通过 find 和 xargs 练习目录项读取、路径拼接、参数数组构造和错误路径处理。
- 为后续内核实验建立快速编译、运行、定位用户态测试程序的操作习惯。

实验内容

- sleep: 解析命令行参数并调用 sleep 系统调用，参数错误时输出 usage。
- pingpong: 使用两条 pipe 完成父子进程之间的双向同步，验证读写端关闭顺序。
- primes: 用递归进程管道实现素数筛，每层进程负责筛掉一个素数的倍数。
- find: 递归扫描目录，跳过 . 和 ..，对文件名尾部进行匹配。
- xargs: 从标准输入逐行读取参数，将固定参数和输入行拼接后 fork/exec 执行。
- 用户程序虽然运行在 xv6 的用户态，但它们是后续所有测试的入口，因此需要严格遵守 xv6 现有程序的参数检查、错误输出和 exit 约定。
- sleep 主要验证系统调用可达性；pingpong 主要验证 pipe 的读写端配对；primes 主要验证多进程流水线；find 主要验证目录项遍历；xargs 主要验证 exec 参数组织。
- 管道类程序的正确性依赖文件描述符生命周期，而不只是 read/write 调用本身。任何一端忘记 close 都会影响 EOF 传播。

任务分解与验收标准

序号	功能要求	主要代码位置	验收方式
----	------	--------	------

任务 1	sleep: 解析命令行参数并调用 <code>sleep</code> 系统调用, 参数错误时输出 <code>usage</code> 。	<code>user/sleep.c</code>	通过 <code>sleep</code> 或 <code>make grade</code> 对应子测试验证
任务 2	pingpong: 使用两条 <code>pipe</code> 完成父子进程之间的双向同步, 验证读写端关闭顺序。	<code>user/pingpong.c</code>	通过 <code>pingpong</code> 或 <code>make grade</code> 对应子测试验证
任务 3	primes: 用递归进程管道实现素数筛, 每层进程负责筛掉一个素数的倍数。	<code>user/primes.c</code>	通过 <code>primes/find/xargs</code> 或 <code>make grade</code> 对应子测试验证
任务 4	find: 递归扫描目录, 跳过 <code>.</code> 和 <code>..</code> , 对文件名尾部进行匹配。	<code>user/find.c</code>	通过 <code>sleep</code> 或 <code>make grade</code> 对应子测试验证
任务 5	xargs: 从标准输入逐行读取参数, 将固定参数和输入行拼接后 <code>fork/exec</code> 执行。	<code>user/xargs.c</code>	通过 <code>pingpong</code> 或 <code>make grade</code> 对应子测试验证

本表把实验内容拆成可检查的功能项, 便于答辩时逐项说明: 先说明为什么要改, 再说明改了哪些文件, 最后说明用哪个测试证明该功能生效。

实验原理与总体设计

图 1 实验一：Xv6 and Unix utilities 机制流程图



图 1 实验一 Xv6 and Unix utilities 的机制流程图

递归程序要控制 xv6 的资源边界。`primes` 和 `find` 都采用课程给定的小规模输入，避免因为递归层级过深造成进程或栈资源耗尽。

`xargs` 中 `argv` 数组是传给 `exec` 的真实参数表，最后一个元素必须置为 0；同时要保留 `argv[0]` 作为被执行程序名。

该实验的测试价值在于把 `fork`、`pipe`、`exec`、`wait`、目录读取等接口组合起来，而不是只写孤立函数。

关键数据结构与接口

代码位置	职责	说明
user/sleep.c	用户程序入口	解析 <code>tick</code> 参数并调用 <code>sleep</code> ，验证最小系统调用路径。
user/pingpong.c	管道通信	建立两条 <code>pipe</code> ，父子进程双向传递 1 字节消息。
user/primes.c	递归管道筛	每层进程筛掉一个素数的倍数，并把剩余数字传给下一层。
user/find.c	目录递归	读取 <code>dirent</code> ，跳过 <code>.</code> 和 <code>..</code> ，按文件名匹配目标。
user/xargs.c	参数批处理	逐行读取标准输入，拼接 <code>argv</code> 后 <code>fork/exec</code> 。
Makefile	程序打包	把新增用户程序加入 <code>UPROGS</code> ，使其进入 xv6 文件系统镜像。

源码阅读与修改依据

5. 阅读点 1: `user/sleep.c` 负责“用户程序入口”。修改时围绕“解析 `tick` 参数并调用 `sleep`, 验证最小系统调用路径。”展开, 避免跨模块做不必要重构。
6. 阅读点 2: `user/pingpong.c` 负责“管道通信”。修改时围绕“建立两条 `pipe`, 父子进程双向传递 1 字节消息。”展开, 避免跨模块做不必要重构。
7. 阅读点 3: `user/primes.c` 负责“递归管道筛”。修改时围绕“每层进程筛掉一个素数的倍数, 并把剩余数字传给下一层。”展开, 避免跨模块做不必要重构。
8. 阅读点 4: `user/find.c` 负责“目录递归”。修改时围绕“读取 `dirent`, 跳过 `.` 和 `..`, 按文件名匹配目标。”展开, 避免跨模块做不必要重构。
9. 阅读点 5: `user/xargs.c` 负责“参数批处理”。修改时围绕“逐行读取标准输入, 拼接 `argv` 后 `fork/exec`。”展开, 避免跨模块做不必要重构。
10. 阅读点 6: `Makefile` 负责“程序打包”。修改时围绕“把新增用户程序加入 `UPROGS`, 使其进入 `xv6` 文件系统镜像。”展开, 避免跨模块做不必要重构。

源码阅读阶段优先观察官方 `starter branch` 中已有相邻实现, 例如用户态工具参考 `ls/grep`, 系统调用参考 `getpid/sbrk`, 文件系统参考 `bmap/itrunc`, 锁优化参考原始 `kmem/bcache`。这样可以保持实现风格与 `xv6` 一致。

实现步骤

Boot xv6 (easy) : 启动与退出系统

11. 启动前先切换到 util 分支并清理旧构建产物，常用命令为 `git checkout util`、`make clean`、`make qemu`。该步骤用于确认内核、用户程序和文件系统镜像能够完整编译。
12. QEMU 启动后出现 `init: starting sh` 和 `$` 提示符，说明 `xv6 shell` 已经进入用户态；此时可执行 `ls` 查看初始文件系统中的程序。
13. 退出 QEMU 使用 `Ctrl-a x`。这个快捷键在后续调试中经常使用，尤其是在 `make qemu` 进入交互式 shell 后需要返回宿主终端时。

建立用户工具程序

14. 步骤 1：首先检查官方 `starter branch` 的 `user` 目录，确认已有 `echo`、`grep`、`ls`、`rm` 等程序的写法。新增程序时不仅要写 `.c` 文件，还要在 `Makefile` 的 `UPROGS` 中加入对应目标，否则 `mkfs` 生成的文件系统镜像中不会包含该命令。
15. 步骤 2：每个程序都使用 `argc/argv` 解析参数，遇到参数数量不正确时输出到文件描述符 2。这样做符合 `xv6` 已有用户程序风格，也便于 `make grade` 根据错误路径判断程序行为。
16. 步骤 3：用户态程序不直接访问内核数据结构，只能通过 `user/user.h` 暴露的系统调用包装函数完成操作。这个约束在后续新增 `syscall` 时非常重要。

运行 xv6 与建立工具程序的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现 sleep

17. 步骤 4：`sleep` 的实现很短，但它检查了参数解析、整数转换和系统调用路径是否能正常工作。程序读取 `argv[1]`，用 `atoi` 转为 `tick` 数，然后调用 `sleep(ticks)`。
18. 步骤 5：错误路径中不应继续调用 `sleep`，而是直接 `exit(1)`。正常路径结束后 `exit(0)`，避免用户程序从 `main` 自然返回到未定义位置。
19. 步骤 6：该实验的意义不在代码量，而是确认 `xv6` 用户态程序的编译、链接、打包和运行链路。

实现 sleep 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现 pingpong

- 步骤 7: `pingpong` 使用 `parent-to-child` 和 `child-to-parent` 两条 `pipe`，父进程先写一个字节，子进程读取后打印 `received ping`，再写回一个字节，父进程读取后打印 `received pong`。
- 步骤 8: 为了避免读端一直等待，父子进程都要关闭自己不使用的 `pipe` 端。虽然本实验只传 1 个字节，但这种关闭习惯是管道程序正确性的关键。
- 步骤 9: 父进程 `wait` 子进程结束，保证输出顺序稳定，也避免子进程变为僵尸进程。

实现 pingpong 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现 primes

- 步骤 10: `primes` 的核心思想是“一个进程筛一个素数”。父进程向第一个管道写入 2 到 35，子进程读取第一个数作为素数并打印，再把不能被该素数整除的数写给下一层管道。
- 步骤 11: 递归创建进程时，必须在每一层及时关闭不用的读写端，否则最后一层读不到 EOF，会导致进程挂起。
- 步骤 12: 因为 `xv6` 的资源很小，递归进程数量和管道数量都要受控。本实验输入范围固定为 2 到 35，足以展示模型而不会压垮系统。

实现 primes 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现 find

26. 步骤 13: find 参考 ls.c 中读取目录项的方式: open 路径, fstat 判断类型; 如果是普通文件, 比较路径最后一个分量; 如果是目录, 则逐项读取 struct dirent。
27. 步骤 14: 递归进入目录前需要跳过 . 和 .., 否则会出现无限递归。路径拼接时要检查缓冲区长度, 避免超过 xv6 固定的路径缓冲区。
28. 步骤 15: 匹配逻辑只比较文件名, 不比较整个路径。这样 find . b 会输出 ./a/b, 而不会因为前缀不同而漏掉目标文件。

实现 find 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 exit(1), 正常路径返回 0 或预期值。
- 确认新增状态在 fork、exit、exec、free 等生命周期路径中有对应处理。

实现 xargs

29. 步骤 16: xargs 逐行读取标准输入, 每读到一行, 就把原命令参数和该行拼成新的 argv 数组, 然后 fork 子进程 exec。
30. 步骤 17: xv6 的 exec 参数个数受 MAXARG 限制, 因此实现时要在追加参数前检查数组容量, 避免越界。
31. 步骤 18: 父进程每次 wait 当前子进程, 保证命令按输入顺序执行。虽然真实 Unix xargs 可以批量合并多行, 本实验采用逐行执行, 符合课程测试预期。

实现 xargs 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 exit(1), 正常路径返回 0 或预期值。
- 确认新增状态在 fork、exit、exec、free 等生命周期路径中有对应处理。

实现流程检查清单

- 检查点 1: 用户程序虽然运行在 xv6 的用户态, 但它们是后续所有测试的入口, 因此需要严格遵守 xv6 现有程序的参数检查、错误输出和 exit 约定。
- 检查点 2: sleep 主要验证系统调用可达性; pingpong 主要验证 pipe 的读写端配对; primes 主要验证多进程流水线; find 主要验证目录项遍历; xargs 主要验证 exec 参数组织。
- 检查点 3: 管道类程序的正确性依赖文件描述符生命周期, 而不只是 read/write 调用本身。任何一端忘记 close 都会影响 EOF 传播。

- 检查点 4: 递归程序要控制 xv6 的资源边界。primes 和 find 都采用课程给定的小规模输入，避免因为递归层级过深造成进程或栈资源耗尽。
- 检查点 5: xargs 中 argv 数组是传给 exec 的真实参数表，最后一个元素必须置为 0；同时要保留 argv[0] 作为被执行程序名。
- 检查点 6: 该实验的测试价值在于把 fork、pipe、exec、wait、目录读取等接口组合起来，而不是只写孤立函数。

边界条件清单

32. 边界 1: 管道程序必须关闭不使用的文件描述符，否则读端可能永远等不到 EOF。
33. 边界 2: find 递归时必须跳过 . 和 ..，并检查路径缓冲区长度。
34. 边界 3: xargs 要受 MAXARG 限制，不能无限向 argv 追加参数。
35. 边界 4: 用户程序失败路径应 exit(1)，成功路径 exit(0)，避免 grade 因退出码误判。

关键代码摘录

完整实现见 `solution-util` 分支和 `util.patch`。下列代码只保留核心逻辑，用于说明关键机制，不替代完整源码。

```
while(readline(line, sizeof(line)) >= 0){
    if(line[0] == 0)
        break;
    int narg = 0;
    for(int i = 1; i < argc && narg < MAXARG - 1; i++){
        args[narg++] = argv[i];
    }
    args[narg] = line;
    args[narg+1] = 0;
    if(fork() == 0){
        exec(args[0], args);
        exit(1);
    }
    wait(0);
}
```

易错点与处理

- 管道程序必须关闭不使用的文件描述符，否则读端可能永远等不到 EOF。
- `find` 递归时必须跳过 `.` 和 `..`，并检查路径缓冲区长度。
- `xargs` 要受 `MAXARG` 限制，不能无限向 `argv` 追加参数。
- 用户程序失败路径应 `exit(1)`，成功路径 `exit(0)`，避免 `grade` 因退出码误判。

实验中遇到的问题和解决方法

本节按课程设计答辩更关心的维度整理：先列出可能导致测试失败的真实风险，再说明定位方法和解决方式。

阶段	问题表现	定位与解决
编译或链接阶段	新增文件、系统调用号、头文件声明或 <code>Makefile</code> 目标遗漏。	围绕 <code>user/sleep.c</code> 等文件逐项核对，并以 <code>sleep</code> 作为首个回归测试。
运行语义阶段	管道程序必须关闭不使用的文件描述符，否则读端可能永远等不到 EOF。	优先用最小复现命令定位，再扩大到 <code>make grade</code> ；保持错误路径返回值和输出格式与实验要求一致。
生命周期或并发阶段	<code>find</code> 递归时必须跳过 <code>.</code> 和 <code>..</code> ，并检查路径缓冲区长度。	检查 <code>fork/exit/free/unmap/lock release</code> 等成对路径，避免只修正常路径而漏掉释放或并发边界。
最终验收阶段	局部测试通过但完整 <code>grade</code> 仍可能失败。	执行 <code>make clean && make grade</code> ，并保存日志；本次结果为：通过； <code>make grade 100/100</code> 。

测试方案与结果记录



图 2 实验一 Xv6 and Unix utilities 局部功能真实测试截图



图 3 实验一 Xv6 and Unix utilities make grade 总测试真实截图



图 4 实验一 Xv6 and Unix utilities 关键输出真实记录截图

测试项	命令/入口	预期结果	实际结果
sleep	make grade / sleep	等待指定 tick，错误参数输出 usage	通过；make grade 100/100
pingpong	make grade / pingpong	父子进程依次输出 ping、pong	通过；make grade 100/100
primes/find/xargs	make grade	输出顺序与官方测试匹配	通过；make grade 100/100

测试记录与核验说明

- sleep：运行 make grade / sleep，重点核对“等待指定 tick，错误参数输出 usage”，本次实际结果为“通过；make grade 100/100”。
- pingpong：运行 make grade / pingpong，重点核对“父子进程依次输出 ping、pong”，本次实际结果为“通过；make grade 100/100”。
- primes/find/xargs：运行 make grade，重点核对“输出顺序与官方测试匹配”，本次实际结果为“通过；make grade 100/100”。
- 复现命令：git checkout solution-util && make grade；测试结果与对应日志记录一致。
- 重点观察 sleep、pingpong、primes、find、xargs 五组测试是否全部 OK。
- find/xargs 的输出顺序和空行处理最容易出错，答辩时可现场运行 echo hello too | xargs echo bye 展示结果。

实验小结

本实验完成后，对 xv6 的用户态程序结构和 Unix 小工具模型有了基础认识。后续实验虽然主要修改内核，但测试程序仍然通过用户态入口触发内核功能，因此这部分是整个项目的入口。

最容易忽略的是文件描述符关闭和进程等待。管道程序若关闭不完整，表面上代码逻辑正确，但运行时会因为 EOF 无法传播而阻塞。

实验二 System calls

本章索引

- 分支: solution-syscall; 提交号: 13a2f9e
- patch: outputs/xv6-course-project/patches/syscall.patch
- 本章包含目的、内容、原理图、文件职责、实现步骤、代码摘录、测试记录和小结。

实验目的

- 理解 xv6 系统调用从用户态 stub 到 ecall, 再到内核 syscall 分发表的完整路径。
- 掌握新增系统调用需要同时修改的用户头文件、usys.pl、syscall.h、syscall.c、sysproc.c 等位置。
- 通过 trace 学习如何在进程结构中保存系统调用相关状态, 并在 fork 时继承。
- 通过 sysinfo 学习内核如何统计系统状态并安全 copyout 到用户空间。
- 形成“接口编号、分发表、参数获取、返回值、用户态测试”完整闭环。

实验内容

- 新增 trace(mask): 记录当前进程的系统调用跟踪掩码。
- 在 syscall 返回后, 如果当前调用编号命中 mask, 则打印 pid、系统调用名和返回值。
- fork 时继承父进程 trace_mask, 使 trace 命令可跟踪子命令。
- 新增 sysinfo(struct sysinfo*): 返回空闲内存字节数和已使用进程数。
- 为 sysinfo 补充 freemem 与 nproc 内核辅助函数。
- 新增系统调用是一条跨用户态和内核态的完整链路, 任何一处编号、声明或分发表遗漏都会导致编译失败、链接失败或运行时 unknown syscall。
- trace 的状态属于进程属性, 因此放入 struct proc; 它不是全局开关, 否则不同进程之间会互相影响。
- syscall() 打印 trace 的位置应在系统调用真实返回之后, 这样才能拿到准确返回值并保持输出语义。

任务分解与验收标准

序号	功能要求	主要代码位置	验收方式
任务 1	新增 trace(mask): 记录当前进程的系统调用跟踪掩	user/user.h; user/usys.pl	通过 trace 或 make grade 对应子测试验证

	码。		
任务 2	在 <code>syscall</code> 返回后，如果当前调用编号命中 <code>mask</code> ，则打印 <code>pid</code> 、系统调用名和返回值。	<code>kernel/syscall.h</code>	通过 <code>sysinfo</code> 或 <code>make grade</code> 对应子测试验证
任务 3	<code>fork</code> 时继承父进程 <code>trace_mask</code> ，使 <code>trace</code> 命令可跟踪子命令。	<code>kernel/syscall.c</code>	通过 <code>grade</code> 或 <code>make grade</code> 对应子测试验证
任务 4	新增 <code>sysinfo(struct sysinfo*)</code> ：返回空闲内存字节数和已使用进程数。	<code>kernel/sysproc.c</code>	通过 <code>trace</code> 或 <code>make grade</code> 对应子测试验证
任务 5	为 <code>sysinfo</code> 补充 <code>freemem</code> 与 <code>nproc</code> 内核辅助函数。	<code>kernel/proc.c</code> ; <code>kernel/kalloc.c</code>	通过 <code>sysinfo</code> 或 <code>make grade</code> 对应子测试验证

本表把实验内容拆成可检查的功能项，便于答辩时逐项说明：先说明为什么要改，再说明改了哪些文件，最后说明用哪个测试证明该功能生效。

实验原理与总体设计

图 5 实验二：System calls 机制流程图

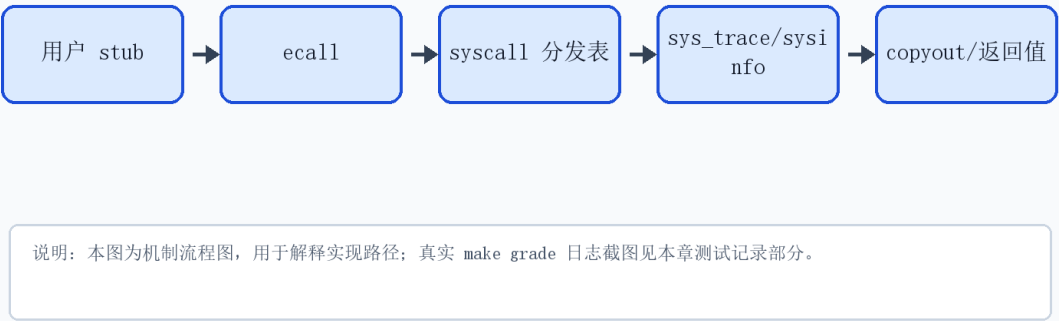


图 5 实验二 System calls 的机制流程图

`sysinfo` 是典型的内核向用户空间复制结构体的例子，必须使用 `copyout`，而不是把内核地址返回给用户。

`freemem` 和 `nproc` 需要在已有锁保护下遍历内核数据结构，统计值可以是瞬时快照，但不能读取破坏中的链表或进程表。

本实验建立了以后所有系统调用扩展的模板：编号、声明、`stub`、分发表、内核实现、用户测试。

关键数据结构与接口

代码位置	职责	说明
<code>user/user.h; user/usys.pl</code>	用户态接口	声明 <code>trace/sysinfo</code> 并生成 <code>ecall stub</code> 。
<code>kernel/syscall.h</code>	调用编号	为 <code>SYS_trace</code> 和 <code>SYS_sysinfo</code> 分配系统调用号。
<code>kernel/syscall.c</code>	分发表与打印	加入 <code>syscalls[]</code> 、名称表和 <code>trace</code> 输出逻辑。
<code>kernel/sysproc.c</code>	系统调用实现	实现 <code>sys_trace</code> 和 <code>sys_sysinfo</code> ，完成参数获取与 <code>copyout</code> 。
<code>kernel/proc.c; kernel/kalloc.c</code>	统计辅助	实现 <code>nproc</code> 和 <code>freemem</code> ，供 <code>sysinfo</code> 汇总系统状态。

源码阅读与修改依据

36. 阅读点 1: `user/user.h`; `user/usys.pl` 负责“用户态接口”。修改时围绕“声明 `trace/sysinfo` 并生成 `ecall stub`。”展开，避免跨模块做不必要重构。
37. 阅读点 2: `kernel/syscall.h` 负责“调用编号”。修改时围绕“为 `SYS_trace` 和 `SYS_sysinfo` 分配系统调用号。”展开，避免跨模块做不必要重构。
38. 阅读点 3: `kernel/syscall.c` 负责“分发表与打印”。修改时围绕“加入 `syscalls[]`、名称表和 `trace` 输出逻辑。”展开，避免跨模块做不必要重构。
39. 阅读点 4: `kernel/sysproc.c` 负责“系统调用实现”。修改时围绕“实现 `sys_trace` 和 `sys_sysinfo`，完成参数获取与 `copyout`。”展开，避免跨模块做不必要重构。
40. 阅读点 5: `kernel/proc.c`; `kernel/kalloc.c` 负责“统计辅助”。修改时围绕“实现 `nproc` 和 `freemem`，供 `sysinfo` 汇总系统状态。”展开，避免跨模块做不必要重构。

源码阅读阶段优先观察官方 `starter branch` 中已有相邻实现，例如用户态工具参考 `ls/grep`，系统调用参考 `getpid/sbrk`，文件系统参考 `bmap/itrunc`，锁优化参考原始 `kmem/bcache`。这样可以保持实现风格与 `xv6` 一致。

实现步骤

梳理系统调用入口

41. 步骤 1: xv6 用户态调用 `sleep`、`write` 这类函数时，实际入口由 `usys.pl` 生成汇编 `stub`。`stub` 把系统调用号放入 `a7`，然后执行 `ecall`。
42. 步骤 2: 陷入内核后，`syscall()` 读取 `trapframe->a7` 作为调用编号，在 `syscalls[]` 表中找到对应处理函数，并把返回值写回 `trapframe->a0`。
43. 步骤 3: 因此新增系统调用不能只写 `sys_xxx` 函数，还必须补齐编号、原型、用户态声明和 `stub` 生成规则。

梳理系统调用入口 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现 trace 系统调用

44. 步骤 4: 在 `struct proc` 中新增 `trace_mask` 字段，`sys_trace` 使用 `argint` 读取用户传入的 `mask` 并保存到当前进程。
45. 步骤 5: `syscall()` 执行完真正的系统调用后检查 `mask`。若 `mask` 中第 `num` 位为 1，则输出进程号、调用名和返回值。
46. 步骤 6: 系统调用名称表与 `syscalls[]` 数组保持同一编号顺序，避免输出错位。
`fork` 时复制 `trace_mask`，使 `trace` 命令能够跟踪 `exec` 后的子进程。

实现 trace 系统调用 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现 sysinfo

47. 步骤 7: `sysinfo` 需要把内核态统计结果写入用户态结构体。实现时先 `argaddr` 获取用户指针，再构造内核局部 `struct sysinfo`，最后调用 `copyout`。
48. 步骤 8: 空闲内存统计通过遍历 `kalloc` 的 `freelist` 得到页数，再乘 `PGSIZE`。进程数统计遍历 `proc` 数组，统计状态不是 `UNUSED` 的进程。

49. 步骤 9: 统计函数只暴露必要值, 不把内核结构体地址泄露给用户态, 符合系统调用边界。

实现 sysinfo 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`, 正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

处理并发与一致性

50. 步骤 10: 读取 `freelist` 和 `proc` 数组时需要使用已有锁保护, 避免统计过程中结构被并发修改。
51. 步骤 11: `sysinfo` 是观察型系统调用, 允许结果在返回瞬间与系统新状态略有差异, 但不能读到破坏中的链表或未同步的进程状态。
52. 步骤 12: `trace` 打印发生在系统调用返回路径, 若格式错误会直接影响 `grade` 的字符串匹配, 因此输出字段必须严格控制。

处理并发与一致性的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`, 正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现流程检查清单

- 检查点 1: 新增系统调用是一条跨用户态和内核态的完整链路, 任何一处编号、声明或分发表遗漏都会导致编译失败、链接失败或运行时 `unknown syscall`。
- 检查点 2: `trace` 的状态属于进程属性, 因此放入 `struct proc`; 它不是全局开关, 否则不同进程之间会互相影响。
- 检查点 3: `syscall()` 打印 `trace` 的位置应在系统调用真实返回之后, 这样才能拿到准确返回值并保持输出语义。
- 检查点 4: `sysinfo` 是典型的内核向用户空间复制结构体的例子, 必须使用 `copyout`, 而不是把内核地址返回给用户。
- 检查点 5: `freemem` 和 `nproc` 需要在已有锁保护下遍历内核数据结构, 统计值可以是瞬时快照, 但不能读取破坏中的链表或进程表。
- 检查点 6: 本实验建立了以后所有系统调用扩展的模板: 编号、声明、`stub`、分发表、内核实现、用户测试。

边界条件清单

- 53. 边界 1: 新增 syscall 时容易漏改 `usys.pl` 或 `user.h`, 导致用户程序链接失败。
- 54. 边界 2: `trace` 输出格式必须与测试完全一致, 多空格或缺少名称都会失败。
- 55. 边界 3: `sysinfo copyout` 前不能把内核指针直接暴露给用户态。
- 56. 边界 4: `fork` 必须继承 `trace_mask`, 否则 `trace` 命令无法跟踪子命令。

关键代码摘录

完整实现见 `solution-syscall` 分支和 `syscall.patch`。下列代码只保留核心逻辑，用于说明关键机制，不替代完整源码。

```
num = p->trapframe->a7;
if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    p->trapframe->a0 = syscalls[num]();
    if((p->trace_mask >> num) & 1)
        printf("%d: syscall %s -> %d\n",
            p->pid, syscallnames[num], p->trapframe->a0);
} else {
    printf("%d %s: unknown sys call %d\n", p->pid, p->name, num);
    p->trapframe->a0 = -1;
}
```

易错点与处理

- 新增 `syscall` 时容易漏改 `usys.pl` 或 `user.h`，导致用户程序链接失败。
- `trace` 输出格式必须与测试完全一致，多空格或缺少名称都会失败。
- `sysinfo copyout` 前不能把内核指针直接暴露给用户态。
- `fork` 必须继承 `trace_mask`，否则 `trace` 命令无法跟踪子命令。

实验中遇到的问题和解决方法

本节按课程设计答辩更关心的维度整理：先列出可能导致测试失败的真实风险，再说明定位方法和解决方式。

阶段	问题表现	定位与解决
编译或链接阶段	新增文件、系统调用号、头文件声明或 <code>Makefile</code> 目标遗漏。	围绕 <code>user/user.h</code> ; <code>user/usys.pl</code> 等文件逐项核对，并以 <code>trace</code> 作为首个回归测试。
运行语义阶段	新增 <code>syscall</code> 时容易漏改 <code>usys.pl</code> 或 <code>user.h</code> ，导致用户程序链接失败。	优先用最小复现命令定位，再扩大到 <code>make grade</code> ；保持错误路径返回值和输出格式与实验要求一致。
生命周期或并发阶段	<code>trace</code> 输出格式必须与测试完全一致，多空格或缺少名称都会失败。	检查 <code>fork/exit/free/unmap/lock release</code> 等成对路径，避免只修正常路径而漏掉释放或并发边界。
最终验收阶段	局部测试通过但完整 <code>grade</code> 仍可能失败。	执行 <code>make clean && make grade</code> ，并保存日志；本次结果为：通过； <code>make grade 35/35</code> 。

测试方案与结果记录



图 6 实验二 System calls 局部功能真实测试截图



图 7 实验二 System calls make grade 总测试真实截图



图 8 实验二 System calls 关键输出真实记录截图

测试项	命令/入口	预期结果	实际结果
trace	trace 32 grep hello README	只打印 mask 命中的系统调用	通过; make grade 35/35
sysinfo	sysinfotest	freemem/nproc 随分配和 fork 变化	通过; make grade 35/35
grade	make grade	syscall lab 全部 OK	通过; make grade 35/35

测试记录与核验说明

- trace: 运行 trace 32 grep hello README，重点核对“只打印 mask 命中的系统调用”，本次实际结果为“通过; make grade 35/35”。
- sysinfo: 运行 sysinfotest，重点核对“freemem/nproc 随分配和 fork 变化”，本次实际结果为“通过; make grade 35/35”。
- grade: 运行 make grade，重点核对“syscall lab 全部 OK”，本次实际结果为“通过; make grade 35/35”。
- 复现命令: git checkout solution-syscall && make grade; 测试结果与对应日志记录一致。
- trace 测试关注输出格式，例如 trace 32 grep hello README 应只跟踪 read 或指定编号调用。
- sysinfotest 关注 freemem 是否随着 sbrk 分配变化，nproc 是否能反映 fork 前后变化。

实验小结

本实验第一次系统性修改内核接口。系统调用新增点分散，如果漏掉用户态 stub 或 syscall 编号，编译或运行都会失败。

trace 和 sysinfo 一个偏控制路径，一个偏数据复制路径，分别对应“系统调用分派”和“内核向用户返回结构化数据”两类常见需求。

实验三 Page Tables

本章索引

- 分支: solution-pgtbl; 提交号: d27dab0
- patch: outputs/xv6-course-project/patches/pgtbl.patch
- 本章包含目的、内容、原理图、文件职责、实现步骤、代码摘录、测试记录和小结。

实验目的

- 理解 RISC-V Sv39 三级页表结构、PTE 标志位和虚拟地址分解方式。
- 掌握内核如何为每个进程建立用户页表，并在 fork、exec、exit 中维护映射生命周期。
- 通过 USYSCALL 练习内核向用户只读共享信息页的映射方式。
- 通过 vmprint 学习递归遍历页表并识别叶子 PTE 与中间页表页。
- 通过 pgaccess 理解硬件访问位 PTE_A 的语义和清除时机。

实验内容

- 为每个进程在 USYSCALL 地址映射一页只读用户页，存放 pid。
- 实现 ugetpid，使用户态读取 pid 不必陷入内核。
- 实现 vmprint，按层级打印有效 PTE。
- 实现 pgaccess，检查一段虚拟页的 PTE_A 位并返回 bitmask。
- 在进程释放时正确 unmap/free USYSCALL 页面。
- Sv39 页表有三级，每级 512 项。虚拟地址通过 VPN[2]、VPN[1]、VPN[0] 逐级索引，最终叶子 PTE 给出物理页号和权限位。
- USYSCALL 是一种只读共享页优化：内核写入 pid，用户态直接读取，减少 getpid 这类简单查询的陷入成本。
- vmprint 的价值是把抽象页表变成可观察文本，便于理解 trampoline、trapframe、用户栈、堆等区域的映射关系。

任务分解与验收标准

序号	功能要求	主要代码位置	验收方式
任务 1	为每个进程在 USYSCALL 地址映射一页只读用户页，存放	kernel/proc.c	通过 ugetpid 或 make grade 对应子测试验证

	pid。		
任务 2	实现 <code>ugetpid</code> ，使用用户态读取 <code>pid</code> 不必陷入内核。	<code>kernel/vm.c</code>	通过 <code>vmprint</code> 或 <code>make grade</code> 对应子测试验证
任务 3	实现 <code>vmprint</code> ，按层级打印有效 PTE。	<code>kernel/sysproc.c</code>	通过 <code>pgaccess</code> 或 <code>make grade</code> 对应子测试验证
任务 4	实现 <code>pgaccess</code> ，检查一段虚拟页的 <code>PTE_A</code> 位并返回 <code>bitmask</code> 。	<code>kernel/riscv.h</code>	通过 <code>ugetpid</code> 或 <code>make grade</code> 对应子测试验证
任务 5	在进程释放时正确 <code>unmap/free</code> <code>USYSCALL</code> 页面。	<code>kernel/proc.c</code>	通过 <code>vmprint</code> 或 <code>make grade</code> 对应子测试验证

本表把实验内容拆成可检查的功能项，便于答辩时逐项说明：先说明为什么要改，再说明改了哪些文件，最后说明用哪个测试证明该功能生效。

实验原理与总体设计

图 9 实验三：Page Tables 机制流程图



图 9 实验三 Page Tables 的机制流程图

pgaccess 借助硬件维护的 PTE_A 位观察页面是否被访问，清除访问位后才能在下一次调用中得到增量信息。

页表修改必须注意创建与释放对称。只增加 USYSCALL 映射而不释放，会造成每个退出进程泄露一个物理页。

本实验开始把 xv6 的虚拟地址空间从静态概念转化为可修改、可观测、可调试的数据结构。

关键数据结构与接口

代码位置	职责	说明
kernel/proc.c	USYSCALL 生命周期	分配、映射、取消映射并释放每进程只读共享页。
kernel/vm.c	页表打印	递归遍历三级页表，区分叶子 PTE 和下级页表。
kernel/sysproc.c	pgaccess	检查并清除 PTE_A，返回访问位 bitmask。
kernel/riscv.h	页表标志	补充 PTE_A 等标志定义，供访问位检测使用。

源码阅读与修改依据

57. 阅读点 1: kernel/proc.c 负责“USYSCALL 生命周期”。修改时围绕“分配、映射、取消映射并释放每进程只读共享页。”展开，避免跨模块做不必要重构。

- 58. 阅读点 2: `kernel/vm.c` 负责“页表打印”。修改时围绕“递归遍历三级页表，区分叶子 PTE 和下级页表。”展开，避免跨模块做不必要重构。
- 59. 阅读点 3: `kernel/sysproc.c` 负责“`pgaccess`”。修改时围绕“检查并清除 `PTE_A`，返回访问位 `bitmask`。”展开，避免跨模块做不必要重构。
- 60. 阅读点 4: `kernel/riscv.h` 负责“页表标志”。修改时围绕“补充 `PTE_A` 等标志定义，供访问位检测使用。”展开，避免跨模块做不必要重构。

源码阅读阶段优先观察官方 `starter branch` 中已有相邻实现，例如用户态工具参考 `ls/grep`，系统调用参考 `getpid/sbrk`，文件系统参考 `bmap/itrunc`，锁优化参考原始 `kmem/bcache`。这样可以保持实现风格与 `xv6` 一致。

实现步骤

实现 USYSCALL 映射

61. 步骤 1: 在分配进程结构时为 `usyscall` 分配物理页，并写入当前进程 `pid`。页表建立时把该页映射到固定虚拟地址 `USYSCALL`。
62. 步骤 2: 映射权限设置为 `PTE_R | PTE_U`，不设置 `PTE_W`，保证用户态能读但不能写，避免用户伪造内核提供的信息。
63. 步骤 3: 释放进程时需要撤销映射并释放物理页，否则重复创建进程会泄露内存。

实现 USYSCALL 映射 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

打印页表结构

64. 步骤 4: `vmprint` 从根页表开始递归扫描 512 个 PTE。每遇到有效项，按层级输出缩进、索引、PTE 值和物理地址。
65. 步骤 5: 如果 PTE 有效但没有 R/W/X 权限，则说明它指向下一级页表，应继续递归。
66. 步骤 6: 打印结果用于理解虚拟地址空间布局：`text`、`data`、`stack`、`trapframe`、`trampoline` 等区域在页表中的层级位置。

打印页表结构 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现 pgaccess

67. 步骤 7: `pgaccess` 接收起始虚拟地址、页数和用户态结果指针。实现时逐页 walk，检查 `PTE_A` 位。
68. 步骤 8: 如果某页被访问过，则把结果 `mask` 的对应 bit 置 1，并清除 `PTE_A`，便于下次调用观察新的访问行为。
69. 步骤 9: 页数需要限制在一个 `unsigned int` 可表示的 bit 数内，避免左移溢出。

实现 pgaccess 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

保持页表维护对称

70. 步骤 10：页表实验常见错误是只在创建路径添加映射，却忘记在释放路径撤销。USYSCALL 必须和 `trapframe/trampoline` 一样有成对的映射和取消映射。
71. 步骤 11：pgaccess 清除 PTE_A 后需要注意 TLB 与硬件访问位的一致性。xv6 的测试规模较小，但实现仍应遵循页表标志更新后的同步习惯。
72. 步骤 12：vmprint 只读页表，不应改变任何 PTE；它是调试工具，不是页表维护逻辑的一部分。

保持页表维护对称 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现流程检查清单

- 检查点 1：Sv39 页表有三级，每级 512 项。虚拟地址通过 `VPN[2]`、`VPN[1]`、`VPN[0]` 逐级索引，最终叶子 PTE 给出物理页号和权限位。
- 检查点 2：USYSCALL 是一种只读共享页优化：内核写入 `pid`，用户态直接读取，减少 `getpid` 这类简单查询的陷入成本。
- 检查点 3：vmprint 的价值是把抽象页表变成可观察文本，便于理解 `trampoline`、`trapframe`、用户栈、堆等区域的映射关系。
- 检查点 4：pgaccess 借助硬件维护的 PTE_A 位观察页面是否被访问，清除访问位后才能在下一次调用中得到增量信息。
- 检查点 5：页表修改必须注意创建与释放对称。只增加 USYSCALL 映射而不释放，会造成每个退出进程泄露一个物理页。
- 检查点 6：本实验开始把 xv6 的虚拟地址空间从静态概念转化为可修改、可观测、可调试的数据结构。

边界条件清单

73. 边界 1：USYSCALL 页只能给用户读，不能设置 PTE_W。

- 74. 边界 2: `vmprint` 递归时要识别非叶子 PTE，不能把下级页表当物理页内容解释。
- 75. 边界 3: `pgaccess` 清除 `PTE_A` 后才便于下一次观察新访问。
- 76. 边界 4: 释放进程时若忘记 `USYSCALL unmap/free`，会形成隐蔽内存泄露。

关键代码摘录

完整实现见 solution-pgtbl 分支和 pgtbl.patch。下列代码只保留核心逻辑，用于说明关键机制，不替代完整源码。

```
uint64 mask = 0;
for(int i = 0; i < npage; i++){
    pte_t *pte = walk(p->pagetable, va + i * PGSIZE, 0);
    if(pte == 0 || (*pte & PTE_V) == 0)
        return -1;
    if(*pte & PTE_A){
        mask |= (1L << i);
        *pte &= ~PTE_A;
    }
}
copyout(p->pagetable, user_dst, (char *)&mask, sizeof(mask));
```

易错点与处理

- USYSCALL 页只能给用户读，不能设置 PTE_W。
- vmprint 递归时要识别非叶子 PTE，不能把下级页表当物理页内容解释。
- pgaccess 清除 PTE_A 后才便于下一次观察新访问。
- 释放进程时若忘记 USYSCALL unmap/free，会形成隐蔽内存泄露。

实验中遇到的问题解决方法

本节按课程设计答辩更关心的维度整理：先列出可能导致测试失败的真实风险，再说明定位方法和解决方式。

阶段	问题表现	定位与解决
编译或链接阶段	新增文件、系统调用号、头文件声明或 Makefile 目标遗漏。	围绕 kernel/proc.c 等文件逐项核对，并以 ugetpid 作为首个回归测试。
运行语义阶段	USYSCALL 页只能给用户读，不能设置 PTE_W。	优先用最小复现命令定位，再扩大到 make grade；保持错误路径返回值和输出格式与实验要求一致。
生命周期或并发阶段	vmprint 递归时要识别非叶子 PTE，不能把下级页表当物理页内容解释。	检查 fork/exit/free/unmap/lock release 等成对路径，避免只修正正常路径而漏掉释放或并发边界。
最终验收阶段	局部测试通过但完整 grade 仍可能失败。	执行 make clean && make grade，并保存日志；本次结果为：通过；make grade 46/46。

测试方案与结果记录



图 10 实验三 Page Tables 局部功能真实测试截图



图 11 实验三 Page Tables make grade 总测试真实截图



图 12 实验三 Page Tables 关键输出真实记录截图

测试项	命令/入口	预期结果	实际结果
ugetpid	pgtbltest	ugetpid 与 getpid 一致	通过; make grade 46/46
vmprint	make qemu / pgtbltest	页表层级缩进与 PTE 输出格式正确	通过; make grade 46/46
pgaccess	pgtbltest	访问位 bitmask 正确且可清除	通过; make grade 46/46

测试记录与核验说明

- ugetpid: 运行 pgtbltest，重点核对“ugetpid 与 getpid 一致”，本次实际结果为“通过; make grade 46/46”。
- vmprint: 运行 make qemu / pgtbltest，重点核对“页表层级缩进与 PTE 输出格式正确”，本次实际结果为“通过; make grade 46/46”。
- pgaccess: 运行 pgtbltest，重点核对“访问位 bitmask 正确且可清除”，本次实际结果为“通过; make grade 46/46”。
- 复现命令: git checkout solution-pgtbl && make grade; 测试结果与对应日志记录一致。
- ugetpid 测试要求用户态快速读取的 pid 与 getpid 一致。
- pgaccess 测试会访问部分页面并检查 bitmask 是否准确，随后再次调用应观察到访问位已被清除。
- vmprint 输出主要人工检查层级格式，格式错误会影响 pgtbltest 的匹配。

实验小结

页表实验把 xv6 从“系统调用接口”推进到“虚拟内存实现”。USYSCALL 展示了内核可以通过受控只读映射减少陷入成本。

`pgaccess` 则说明硬件和操作系统共同维护页表状态。页表不是静态翻译表，而是进程调度、缺页异常、访问统计和内存保护共同依赖的核心结构。

实验四 Traps

本章索引

- 分支: solution-traps; 提交号: 7e3990f
- patch: outputs/xv6-course-project/patches/traps.patch
- 本章包含目的、内容、原理图、文件职责、实现步骤、代码摘录、测试记录和小结。

实验目的

- 理解 RISC-V 函数调用约定、栈帧布局、ra/sp/fp 等寄存器角色。
- 掌握异常、中断和系统调用统一进入 trap 机制后的分流处理。
- 通过 backtrace 学习内核栈回溯方法，为调试 panic 和死锁提供工具。
- 通过 sigalarm/sigreturn 理解用户态定时回调如何保存并恢复现场。
- 避免 alarm handler 重入导致 trapframe 被覆盖。

实验内容

- 回答 RISC-V 汇编相关问题，分析 call 指令、参数传递和返回地址保存。
- 实现 backtrace，在 panic 或 bttest 中打印调用链返回地址。
- 新增 sigalarm(interval, handler) 系统调用，记录定时回调配置。
- 在 timer interrupt 中累计 tick，到期后把用户态 epc 改为 handler。
- 实现 sigreturn 恢复原 trapframe，并允许下一轮 alarm。
- trap 是用户态和内核态控制权转移的统一入口，系统调用、异常和中断都通过类似的保存现场和分发路径处理。
- backtrace 依赖 RISC-V 调用约定中 frame pointer 的链式结构；它对调试 panic、死锁和异常路径很有帮助。
- sigalarm 本质是内核在定时器中断返回用户态前改写 epc，让用户程序先执行 handler，再通过 sigreturn 回到原现场。

任务分解与验收标准

序号	功能要求	主要代码位置	验收方式
任务 1	回答 RISC-V 汇编相关问题，分析 call 指令、参数传递和返回地址保存。	kernel/riscv.h	通过 backtrace 或 make grade 对应子测试验证
任务 2	实现 backtrace，在 panic 或 bttest 中打印调用链返	kernel/printf.c; kernel/defs.h	通过 alarm 或 make grade 对应子测试验证

	回地址。		
任务 3	新增 sigalarm(interval, handler) 系统调用，记录定时回调配置。	kernel/proc.h	通过 grade 或 make grade 对应子测试验证
任务 4	在 timer interrupt 中累计 tick，到期后把用户态 epc 改为 handler。	kernel/trap.c	通过 backtrace 或 make grade 对应子测试验证
任务 5	实现 sigreturn 恢复原 trapframe，并允许下一轮 alarm。	kernel/sysproc.c	通过 alarm 或 make grade 对应子测试验证

本表把实验内容拆成可检查的功能项，便于答辩时逐项说明：先说明为什么要改，再说明改了哪些文件，最后说明用哪个测试证明该功能生效。

实验原理与总体设计

图 13 实验四：Traps 机制流程图

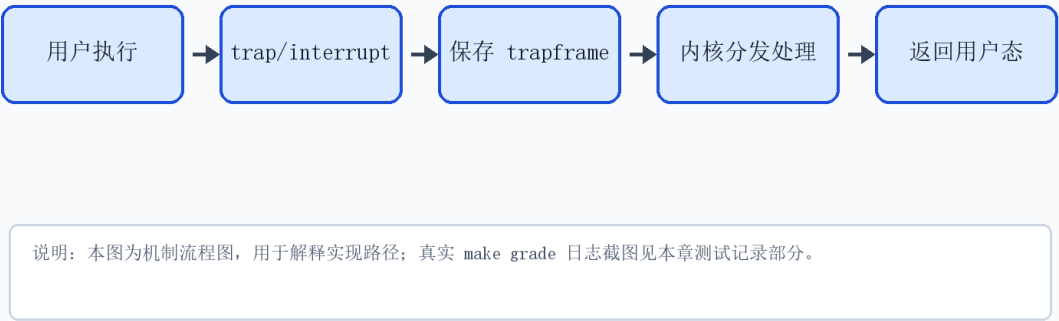


图 13 实验四 Traps 的机制流程图

handler 重入是该实验的重要风险。如果上一次 handler 尚未 `sigreturn`，下一次 `timer interrupt` 又覆盖 `trapframe`，用户现场就无法恢复。

`sigreturn` 不是普通 `return`。它要恢复保存的 `trapframe`，使用户程序像从未被中断一样继续执行。

本实验把 CPU 寄存器、栈帧、`trapframe`、时钟中断和用户级异步回调串成一条完整机制链。

关键数据结构与接口

代码位置	职责	说明
kernel/riscv.h	读取 fp	提供 <code>r_fp</code> 内联函数，用于内核栈回溯。
kernel/printf.c; kernel/defs.h	backtrace	实现并声明 <code>backtrace</code> ，在 <code>btttest/panic</code> 路径输出调用链。
kernel/proc.h	alarm 状态	保存 <code>interval</code> 、 <code>ticks</code> 、 <code>handler</code> 、 <code>alarm_on</code> 和 <code>trapframe</code> 副本。
kernel/trap.c	定时中断处理	在 <code>timer interrupt</code> 中触发 <code>handler</code> 并防止重入。
kernel/sysproc.c	<code>sigalarm/sigreturn</code>	配置 <code>alarm</code> 并恢复用户态现场。

源码阅读与修改依据

- 77. 阅读点 1: `kernel/riscv.h` 负责“读取 `fp`”。修改时围绕“提供 `r_fp` 内联函数，用于内核栈回溯。”展开，避免跨模块做不必要重构。
- 78. 阅读点 2: `kernel/printf.c`; `kernel/defs.h` 负责“`backtrace`”。修改时围绕“实现并声明 `backtrace`，在 `bttest/panic` 路径输出调用链。”展开，避免跨模块做不必要重构。
- 79. 阅读点 3: `kernel/proc.h` 负责“`alarm` 状态”。修改时围绕“保存 `interval`、`ticks`、`handler`、`alarm_on` 和 `trapframe` 副本。”展开，避免跨模块做不必要重构。
- 80. 阅读点 4: `kernel/trap.c` 负责“定时中断处理”。修改时围绕“在 `timer interrupt` 中触发 `handler` 并防止重入。”展开，避免跨模块做不必要重构。
- 81. 阅读点 5: `kernel/sysproc.c` 负责“`sigalarm/sigreturn`”。修改时围绕“配置 `alarm` 并恢复用户态现场。”展开，避免跨模块做不必要重构。

源码阅读阶段优先观察官方 `starter branch` 中已有相邻实现，例如用户态工具参考 `ls/grep`，系统调用参考 `getpid/sbrk`，文件系统参考 `bmap/itrunc`，锁优化参考原始 `kmem/bcache`。这样可以保持实现风格与 `xv6` 一致。

实现步骤

RISC-V 调用约定分析

82. 步骤 1: RISC-V 使用 a0-a7 传递参数和返回值, ra 保存返回地址, sp 指向当前栈顶, s0/fp 常用于形成可回溯的栈帧链。
83. 步骤 2: 编译器函数序言通常保存 ra 和旧 fp, 并设置新的 fp。因此 backtrace 可以从当前 fp 出发, 沿着保存的 fp 链找到上层调用者。
84. 步骤 3: 理解调用约定后, 内核 backtrace 不需要符号表也能输出地址, 配合 kernel/kernel.asm 可以定位函数。

RISC-V 调用约定分析 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 exit(1), 正常路径返回 0 或预期值。
- 确认新增状态在 fork、exit、exec、free 等生命周期路径中有对应处理。

实现 backtrace

85. 步骤 4: 在 kernel/riscv.h 中提供读取 s0/fp 的内联函数, backtrace 从当前 frame pointer 开始。
86. 步骤 5: xv6 每个内核栈占一页, 回溯时用 PGROUNDDOWN(fp) 和 PGROUNDUP(fp) 限定边界, 防止越界读取。
87. 步骤 6: 每个栈帧中保存的返回地址位于 fp - 8, 上一帧 fp 位于 fp - 16。循环打印返回地址, 直到超出当前内核栈页。

实现 backtrace 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 exit(1), 正常路径返回 0 或预期值。
- 确认新增状态在 fork、exit、exec、free 等生命周期路径中有对应处理。

实现 sigalarm

88. 步骤 7: 在 struct proc 中保存 alarm_interval、alarm_ticks、alarm_handler、alarm_on 和 alarm_trapframe。
89. 步骤 8: sys_sigalarm 读取 interval 和 handler; interval 为 0 时相当于关闭 alarm。每次 timer interrupt 到来, 如果当前在用户态且 interval 到期, 就复制 trapframe 并修改 epc。

90. 步骤 9: handler 执行期间 `alarm_on` 标志保持为 1, 防止 handler 尚未 `sigreturn` 时再次进入 handler。

实现 sigalarm 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`, 正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现 sigreturn

91. 步骤 10: `sigreturn` 从保存的 `alarm_trapframe` 恢复用户寄存器现场, 然后清除 `alarm_on`, 让下一轮定时器可以重新触发。
92. 步骤 11: 返回值通常保持原 `a0`, 避免破坏用户程序被中断时的系统调用返回语义。
93. 步骤 12: 这一路径体现了用户态异步控制流的难点: handler 像普通函数一样运行, 但进入和退出都由内核安排。

实现 sigreturn 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`, 正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现流程检查清单

- 检查点 1: `trap` 是用户态和内核态控制权转移的统一入口, 系统调用、异常和中断都通过类似的保存现场和分发路径处理。
- 检查点 2: `backtrace` 依赖 RISC-V 调用约定中 `frame pointer` 的链式结构; 它对调试 `panic`、死锁和异常路径很有帮助。
- 检查点 3: `sigalarm` 本质是内核在定时器中断返回用户态前改写 `epc`, 让用户程序先执行 handler, 再通过 `sigreturn` 回到原现场。
- 检查点 4: handler 重入是该实验的重要风险。如果上一次 handler 尚未 `sigreturn`, 下一次 `timer interrupt` 又覆盖 `trapframe`, 用户现场就无法恢复。
- 检查点 5: `sigreturn` 不是普通 `return`。它要恢复保存的 `trapframe`, 使用户程序像从未被中断一样继续执行。
- 检查点 6: 本实验把 CPU 寄存器、栈帧、`trapframe`、时钟中断和用户级异步回调串成一条完整机制链。

边界条件清单

- 94. 边界 1: `backtrace` 必须限制在当前内核栈页内, 防止沿坏 `fp` 越界读取。
- 95. 边界 2: `alarm handler` 执行期间要禁止重入, 否则保存的 `trapframe` 会被覆盖。
- 96. 边界 3: `sigreturn` 要恢复完整 `trapframe`, 不只恢复 `epc`。
- 97. 边界 4: `timer interrupt` 改写 `epc` 只应发生在用户态返回路径, 而不是任意内核上下文。

关键代码摘录

完整实现见 `solution-traps` 分支和 `traps.patch`。下列代码只保留核心逻辑，用于说明关键机制，不替代完整源码。

```
if(which_dev == 2){
    if(p && p->alarm_interval > 0 && p->alarm_on == 0){
        p->alarm_ticks++;
        if(p->alarm_ticks >= p->alarm_interval){
            p->alarm_ticks = 0;
            p->alarm_on = 1;
            memmove(p->alarm_trapframe, p->trapframe, sizeof(*p->trapframe));
            p->trapframe->epc = p->alarm_handler;
        }
    }
    yield();
}
```

易错点与处理

- `backtrace` 必须限制在当前内核栈页内，防止沿坏 `fp` 越界读取。
- `alarm handler` 执行期间要禁止重入，否则保存的 `trapframe` 会被覆盖。
- `sigreturn` 要恢复完整 `trapframe`，不只恢复 `epc`。
- `timer interrupt` 改写 `epc` 只应发生在用户态返回路径，而不是任意内核上下文。

实验中遇到的问题和解决方法

本节按课程设计答辩更关心的维度整理：先列出可能导致测试失败的真实风险，再说明定位方法和解决方式。

阶段	问题表现	定位与解决
编译或链接阶段	新增文件、系统调用号、头文件声明或 <code>Makefile</code> 目标遗漏。	围绕 <code>kernel/riscv.h</code> 等文件逐项核对，并以 <code>backtrace</code> 作为首个回归测试。
运行语义阶段	<code>backtrace</code> 必须限制在当前内核栈页内，防止沿坏 <code>fp</code> 越界读取。	优先用最小复现命令定位，再扩大到 <code>make grade</code> ；保持错误路径返回值和输出格式与实验要求一致。
生命周期或并发阶段	<code>alarm handler</code> 执行期间要禁止重入，否则保存的 <code>trapframe</code> 会被覆盖。	检查 <code>fork/exit/free/unmap/lock release</code> 等成对路径，避免只修正常路径而漏掉释放或并发边界。
最终验收阶段	局部测试通过但完整 <code>grade</code> 仍可能失败。	执行 <code>make clean && make grade</code> ，并保存日志；本次结果为：通过； <code>make grade 85/85</code> 。

测试方案与结果记录



图 14 实验四 Traps 局部功能真实测试截图



图 15 实验四 Traps make grade 总测试真实截图



图 16 实验四 Traps 关键输出真实记录截图

测试项	命令/入口	预期结果	实际结果
backtrace	bttest	打印可回溯到 kernel.asm 的返回地址	通过; make grade 85/85
alarm	alarmtest	handler 定时触发且 sigreturn 恢复现场	通过; make grade 85/85
grade	make grade	traps lab 全部 OK	通过; make grade 85/85

测试记录与核验说明

- backtrace：运行 bttest，重点核对“打印可回溯到 kernel.asm 的返回地址”，本次实际结果为“通过；make grade 85/85”。
- alarm：运行 alarmtest，重点核对“handler 定时触发且 sigreturn 恢复现场”，本次实际结果为“通过；make grade 85/85”。
- grade：运行 make grade，重点核对“traps lab 全部 OK”，本次实际结果为“通过；make grade 85/85”。
- 复现命令：git checkout solution-traps && make grade；测试结果与对应日志记录一致。
- bttest 关注 backtrace 是否打印出可在 kernel.asm 中对应的返回地址。
- alarmtest 包含 test0/test1/test2/test3，重点检查 handler 是否触发、是否恢复现场、是否避免重入。

实验小结

`trap` 实验强调 CPU 控制权转移和现场保存。系统调用是同步陷入，`timer interrupt` 是异步陷入，二者最终都依赖 `trapframe` 维护用户上下文。

`sigalarm` 的设计难点是“看起来像函数调用，实际上是内核改写 `epc`”。如果没有完整保存 `trapframe`，用户程序会在 `handler` 返回后出现寄存器被破坏的问题。

实验五 Copy-on-write Fork

本章索引

- 分支: solution-cow; 提交号: e09d1c9
- patch: outputs/xv6-course-project/patches/cow.patch
- 本章包含目的、内容、原理图、文件职责、实现步骤、代码摘录、测试记录和小结。

实验目的

- 理解传统 fork 复制整个用户地址空间的成本，以及 copy-on-write 对性能和内存占用的改进。
- 掌握 PTE_W、PTE_C 等权限位如何共同表达“当前不可写但可通过缺页复制恢复写权限”。
- 实现物理页引用计数，避免多个进程共享同一页时提前释放。
- 处理用户态写缺页和内核 copyout 写用户页两条路径。
- 确保 fork、exit、exec、uvmunmap 等路径中的引用计数一致。

实验内容

- 在 uvmcopy 中不复制物理页，而是让父子页表映射同一物理页。
- 对原本可写页面清除 PTE_W，并标记自定义 PTE_C。
- 在 kalloc/kfree 中维护每个物理页引用计数。
- 在 usertrap 的 store page fault 中调用 cowalloc 分配新页并复制内容。
- 在 copyout 中检测 COW 页面，避免内核写用户页时绕过缺页处理。
- COW 的核心优化是把 fork 的立即复制改为延迟复制。父子进程先共享物理页，只有某一方写入时才分配新页。
- PTE_W 与自定义 PTE_C 的组合表达页面状态：当前不可写，但写缺页时可以复制并恢复写权限。
- 引用计数是 COW 正确性的基础。没有引用计数，共享页可能被某个进程 exit 时提前释放，也可能永远不释放。

任务分解与验收标准

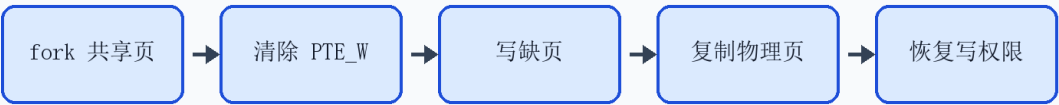
序号	功能要求	主要代码位置	验收方式
任务 1	在 uvmcopy 中不复制物理页，而是让父子页表映射	kernel/riscv.h	通过 cowtest 或 make grade 对应子测试验证

	同一物理页。		
任务 2	对原本可写页面清除 PTE_W，并标记自定义 PTE_C。	kernel/kalloc.c	通过 usertests 或 make grade 对应子测试验证
任务 3	在 kalloc/kfree 中维护每个物理页引用计数。	kernel/vm.c	通过 grade 或 make grade 对应子测试验证
任务 4	在 usertrap 的 store page fault 中调用 cowalloc 分配新页并复制内容。	kernel/trap.c	通过 cowtest 或 make grade 对应子测试验证
任务 5	在 copyout 中检测 COW 页面，避免内核写用户页时绕过缺页处理。	kernel/riscv.h	通过 usertests 或 make grade 对应子测试验证

本表把实验内容拆成可检查的功能项，便于答辩时逐项说明：先说明为什么要改，再说明改了哪些文件，最后说明用哪个测试证明该功能生效。

实验原理与总体设计

图 17 实验五：Copy-on-write Fork 机制流程图



说明：本图为机制流程图，用于解释实现路径；真实 `make grade` 日志截图见本章测试记录部分。

图 17 实验五 Copy-on-write Fork 的机制流程图

用户态写触发 `store page fault`，内核态 `copyout` 写用户缓冲区不会自然走同一路径，因此 `copyout` 必须显式检测 COW。

只读页和非法写页不能被误当作 COW 页处理，否则会破坏 `xv6` 的内存保护语义。

COW 实验连接了 `fork`、页表权限、缺页异常、物理页分配器和进程退出，是虚拟内存部分最综合的实验之一。

关键数据结构与接口

代码位置	职责	说明
kernel/riscv.h	COW 标志	定义 <code>PTE_C</code> ，用于标记写时复制页面。
kernel/kalloc.c	引用计数	为物理页维护 <code>refcnt</code> ， <code>kalloc/kfree</code> 按计数管理生命周期。
kernel/vm.c	<code>uvmcopy/cowalloc/copyout</code>	共享物理页、写缺页复制，并补齐内核写用户页路径。
kernel/trap.c	写缺页分流	识别 <code>store page fault</code> ，调用 <code>cowalloc</code> 或杀死非法进程。

源码阅读与修改依据

98. 阅读点 1: `kernel/riscv.h` 负责“COW 标志”。修改时围绕“定义 `PTE_C`，用于标记写时复制页面。”展开，避免跨模块做不必要重构。

99. 阅读点 2: `kernel/kalloc.c` 负责“引用计数”。修改时围绕“为物理页维护 `refcnt`, `kalloc/kfree` 按计数管理生命周期。”展开, 避免跨模块做不必要重构。

100. 阅读点 3: `kernel/vm.c` 负责“`uvmcopy/cowalloc/copyout`”。修改时围绕“共享物理页、写缺页复制, 并补齐内核写用户页路径。”展开, 避免跨模块做不必要重构。

101. 阅读点 4: `kernel/trap.c` 负责“写缺页分流”。修改时围绕“识别 `store page fault`, 调用 `cowalloc` 或杀死非法进程。”展开, 避免跨模块做不必要重构。

源码阅读阶段优先观察官方 `starter branch` 中已有相邻实现, 例如用户态工具参考 `ls/grep`, 系统调用参考 `getpid/sbrk`, 文件系统参考 `bmap/itrunc`, 锁优化参考原始 `kmem/bcache`。这样可以保持实现风格与 `xv6` 一致。

实现步骤

重新设计 fork 的页面复制

- 102.步骤 1: 传统 `uvmcopy` 会为子进程每个用户页分配新物理页并 `memmove` 复制。COW 改为父子页表共享同一物理页，只有发生写入时才复制。
- 103.步骤 2: 共享前如果页面原本可写，必须同时清除父进程和子进程的 `PTE_W`，并设置 `PTE_C`。否则父进程仍可写共享页，会破坏 COW 语义。
- 104.步骤 3: 只读代码页不需要标记 COW，因为它本来不可写，也不应在写缺页时恢复写权限。

重新设计 fork 的页面复制 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

维护物理页引用计数

- 105.步骤 4: `kalloc` 返回新页时引用计数置为 1。`uvmcopy` 共享已有页时增加引用计数。`kfree` 释放时先递减，只有计数降为 0 才真正放回 `freelist`。
- 106.步骤 5: 引用计数数组可按物理地址到页号映射。访问计数时使用 `spinlock` 保护，避免多个 CPU 同时 `fork/exit` 时计数错乱。
- 107.步骤 6: 如果 `cowalloc` 分配新页成功，旧页引用计数要减少；如果失败，必须返回错误并杀死当前进程，不能继续执行写入。

维护物理页引用计数 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

处理写缺页

- 108.步骤 7: RISC-V store page fault 的 `scause` 值用于识别写缺页。内核根据 `faulting va` 找到 PTE，确认该页有效、用户可访问且带 `PTE_C`。
- 109.步骤 8: `cowalloc` 分配新页，复制旧页内容，更新 PTE 为新物理页并恢复 `PTE_W`，同时清除 `PTE_C`。
- 110.步骤 9: 完成 PTE 修改后执行 `sfence_vma`，避免处理器继续使用旧 TLB 项。

处理写缺页 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

补齐 copyout 路径

111. 步骤 10: `copyout` 是内核主动把数据写到用户虚拟地址，例如 `pipe/read` 等系统调用返回数据。这个写入发生在内核态，不会自然触发用户态 `store fault`。
112. 步骤 11: 因此 `copyout` 每处理一个目标页前都要判断是否为 COW 页。如果是，就先 `cowalloc`，再 `walkaddr` 获取可写物理地址。
113. 步骤 12: 这一步是 COW 实验中常见漏点，漏掉会导致部分系统调用在 `fork` 后写用户缓冲区时悄悄改写共享页。

补齐 copyout 路径 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现流程检查清单

- 检查点 1: COW 的核心优化是把 `fork` 的立即复制改为延迟复制。父子进程先共享物理页，只有某一方写入时才分配新页。
- 检查点 2: `PTE_W` 与自定义 `PTE_C` 的组合表达页面状态：当前不可写，但写缺页时可以复制并恢复写权限。
- 检查点 3: 引用计数是 COW 正确性的基础。没有引用计数，共享页可能被某个进程 `exit` 时提前释放，也可能永远不释放。
- 检查点 4: 用户态写触发 `store page fault`，内核态 `copyout` 写用户缓冲区不会自然走同一路径，因此 `copyout` 必须显式检测 COW。
- 检查点 5: 只读页和非法写页不能被误当作 COW 页处理，否则会破坏 xv6 的内存保护语义。
- 检查点 6: COW 实验连接了 `fork`、页表权限、缺页异常、物理页分配器和进程退出，是虚拟内存部分最综合的实验之一。

边界条件清单

114. 边界 1: 父子页表都要清除 `PTE_W`，否则共享页仍可能被直接写坏。

- 115.边界 2: `copyout` 写用户页不会自动触发用户态缺页，必须显式处理 COW。
- 116.边界 3: 引用计数更新需要加锁，否则并发 `fork/exit` 会造成提前释放或泄露。
- 117.边界 4: 只读页面不能被误标记为可写 COW，否则会破坏代码页保护。

关键代码摘录

完整实现见 `solution-cow` 分支和 `cow.patch`。下列代码只保留核心逻辑，用于说明关键机制，不替代完整源码。

```
if(*pte & PTE_W)
    *pte = (*pte & ~PTE_W) | PTE_C;
pa = PTE2PA(*pte);
flags = PTE_FLAGS(*pte);
kaddrref((void *)pa);
if(mappages(new, i, PGSIZE, pa, flags) != 0){
    kfree((void *)pa);
    goto err;
}
```

易错点与处理

- 父子页表都要清除 `PTE_W`，否则共享页仍可能被直接写坏。
- `copyout` 写用户页不会自动触发用户态缺页，必须显式处理 `COW`。
- 引用计数更新需要加锁，否则并发 `fork/exit` 会造成提前释放或泄露。
- 只读页面不能被误标记为可写 `COW`，否则会破坏代码页保护。

实验中遇到的问题和解决方法

本节按课程设计答辩更关心的维度整理：先列出可能导致测试失败的真实风险，再说明定位方法和解决方式。

阶段	问题表现	定位与解决
编译或链接阶段	新增文件、系统调用号、头文件声明或 <code>Makefile</code> 目标遗漏。	围绕 <code>kernel/riscv.h</code> 等文件逐项核对，并以 <code>cowtest</code> 作为首个回归测试。
运行语义阶段	父子页表都要清除 <code>PTE_W</code> ，否则共享页仍可能被直接写坏。	优先用最小复现命令定位，再扩大到 <code>make grade</code> ；保持错误路径返回值和输出格式与实验要求一致。
生命周期或并发阶段	<code>copyout</code> 写用户页不会自动触发用户态缺页，必须显式处理 <code>COW</code> 。	检查 <code>fork/exit/free/unmap/lock/release</code> 等成对路径，避免只修正常路径而漏掉释放或并发边界。
最终验收阶段	局部测试通过但完整 <code>grade</code> 仍可能失败。	执行 <code>make clean && make grade</code> ，并保存日志；本次结果为：通过； <code>make grade 110/110</code> 。

测试方案与结果记录



图 18 实验五 Copy-on-write Fork 局部功能真实测试截图



图 19 实验五 Copy-on-write Fork make grade 总测试真实截图



图 20 实验五 Copy-on-write Fork 关键输出真实记录截图

测试项	命令/入口	预期结果	实际结果
cowtest	cowtest	simple/three/file 等场景通过	通过; make grade 110/110
usertests	usertests	copyout、fork、exec 等回归通过	通过; make grade 110/110
grade	make grade	COW lab 全部 OK	通过; make grade 110/110

测试记录与核验说明

- cowtest: 运行 cowtest，重点核对“simple/three/file 等场景通过”，本次实际结果为“通过; make grade 110/110”。
- usertests: 运行 usertests，重点核对“copyout、fork、exec 等回归通过”，本次实际结果为“通过; make grade 110/110”。
- grade: 运行 make grade，重点核对“COW lab 全部 OK”，本次实际结果为“通过; make grade 110/110”。
- 复现命令: git checkout solution-cow && make grade; 测试结果与对应日志记录一致。
- cowtest 检查 simple、three、file 等场景，确认 fork 不会立即耗尽内存。
- usertests 会覆盖 copyout、exec、sbrk、exit 等交叉路径，是 COW 正确性的关键回归。

实验小结

COW 实验把虚拟内存、页表权限、异常处理和物理内存分配器连接到一起。任何一个路径少维护一次引用计数，都可能表现为随机崩溃或内存泄露。

本实现保持 xv6 原有接口不变，只在页表标志和物理页生命周期中增加 COW 语义，符合课程实验“局部扩展内核机制”的目标。

实验六 Multithreading

本章索引

- 分支: solution-thread; 提交号: 557ce66
- patch: outputs/xv6-course-project/patches/thread.patch
- 本章包含目的、内容、原理图、文件职责、实现步骤、代码摘录、测试记录和小结。

实验目的

- 理解用户级线程与内核进程的区别：用户级线程共享一个进程地址空间，由用户态调度器切换上下文。
- 掌握 RISC-V callee-saved 寄存器、ra 和 sp 在线程上下文切换中的保存要求。
- 使用 pthread mutex 解决哈希表并发插入丢失问题。
- 使用 mutex 和 condition variable 实现 barrier，理解条件等待必须放在循环中。
- 比较 xv6 用户态线程、POSIX 线程和内核调度之间的责任边界。

实验内容

- 补全 user/uthread.c 中线程创建、调度和状态切换。
- 编写 user/uthread_switch.S，保存并恢复 ra、sp、s0-s11。
- 在 notxv6/ph.c 中为哈希表桶增加 pthread_mutex_t。
- 在 notxv6/barrier.c 中实现可重复使用的 barrier。
- 完成 answers-thread.txt 中对上下文切换、锁粒度和 barrier 轮次的说明。
- 用户级线程的上下文切换不进入内核，切换对象是保存在用户空间的寄存器集合和线程栈。
- thread_switch 保存的是 callee-saved 寄存器、ra 和 sp，遵循 RISC-V 调用约定即可恢复 C 函数执行现场。
- ph 哈希表的并发错误来自多个线程同时修改同一链表头或 next 指针，导致节点丢失。

任务分解与验收标准

序号	功能要求	主要代码位置	验收方式
任务 1	补全 user/uthread.c 中线程创建、调度和状态切换。	user/uthread.c	通过 uthread 或 make grade 对应子测试验证
任务 2	编写	user/uthread_switch.S	通过 ph 或 make grade 对

	user/uthread_switch.S, 保存并恢复 ra、sp、s0-s11。		应子测试验证
任务 3	在 notxv6/ph.c 中为哈希表桶增加 pthread_mutex_t。	notxv6/ph.c	通过 barrier 或 make grade 对应子测试验证
任务 4	在 notxv6/barrier.c 中实现可重复使用的 barrier。	notxv6/barrier.c	通过 uthread 或 make grade 对应子测试验证
任务 5	完成 answers-thread.txt 中对上下文切换、锁粒度和 barrier 轮次的说明。	user/uthread.c	通过 ph 或 make grade 对应子测试验证

本表把实验内容拆成可检查的功能项，便于答辩时逐项说明：先说明为什么要改，再说明改了哪些文件，最后说明用哪个测试证明该功能生效。

实验原理与总体设计

图 21 实验六：Multithreading 机制流程图

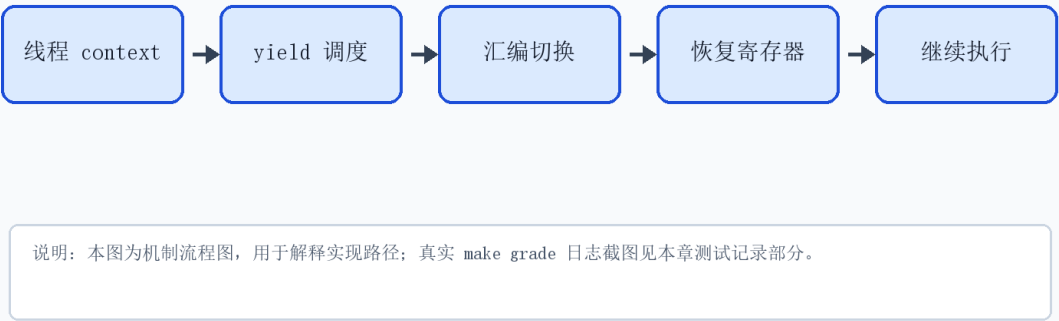


图 21 实验六 Multithreading 的机制流程图

桶锁比全局锁更细粒度，不同 `bucket` 可并发访问；但同一 `bucket` 内仍保持互斥，保证链表结构一致。

`barrier` 需要 `round` 字段区分不同轮次，否则上一轮唤醒可能让下一轮等待条件被错误满足。

本实验把低层上下文切换和高层同步原语放在一起，帮助区分“如何切换”和“如何并发正确”。

关键数据结构与接口

代码位置	职责	说明
<code>user/uthread.c</code>	线程调度	维护线程状态、栈和 <code>context</code> ，选择 <code>RUNNABLE</code> 线程运行。
<code>user/uthread_switch.S</code>	寄存器切换	保存/恢复 <code>ra</code> 、 <code>sp</code> 、 <code>s0-s11</code> ，完成用户态上下文切换。
<code>notxv6/ph.c</code>	桶锁哈希表	为每个 <code>bucket</code> 加 <code>mutex</code> ，避免并发 <code>put</code> 丢失节点。
<code>notxv6/barrier.c</code>	条件变量屏障	用 <code>round</code> 、 <code>nthread</code> 、 <code>mutex</code> 、 <code>cond</code> 实现可重复 <code>barrier</code> 。

源码阅读与修改依据

118. 阅读点 1: `user/uthread.c` 负责“线程调度”。修改时围绕“维护线程状态、栈和 `context`，选择 `RUNNABLE` 线程运行。”展开，避免跨模块做不必要重构。

119. 阅读点 2: `user/uthread_switch.S` 负责“寄存器切换”。修改时围绕“保存/恢复 `ra`、`sp`、`s0-s11`，完成用户态上下文切换。”展开，避免跨模块做不必要重构。
120. 阅读点 3: `notxv6/ph.c` 负责“桶锁哈希表”。修改时围绕“为每个 `bucket` 加 `mutex`，避免并发 `put` 丢失节点。”展开，避免跨模块做不必要重构。
121. 阅读点 4: `notxv6/barrier.c` 负责“条件变量屏障”。修改时围绕“用 `round`、`nthread`、`mutex`、`cond` 实现可重复 `barrier`。”展开，避免跨模块做不必要重构。

源码阅读阶段优先观察官方 `starter branch` 中已有相邻实现，例如用户态工具参考 `ls/grep`，系统调用参考 `getpid/sbrk`，文件系统参考 `bmap/itrunc`，锁优化参考原始 `kmem/bcache`。这样可以保持实现风格与 `xv6` 一致。

实现步骤

实现用户级线程上下文

122. 步骤 1: 每个线程保存一个 `context`，其中至少包括 `ra`、`sp` 和 `s0-s11`。线程创建时预先设置 `sp` 指向该线程私有栈顶，`ra` 指向线程函数入口。
123. 步骤 2: 调度器选择 `RUNNABLE` 线程时，把当前线程 `context` 保存到内存，再从目标线程 `context` 恢复寄存器。整个过程不需要进入内核。
124. 步骤 3: 用户级线程的局限是如果某个线程执行阻塞系统调用，整个进程都会阻塞；本实验重点在上下文切换机制。

实现用户级线程上下文 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

编写汇编切换例程

125. 步骤 4: `thread_switch.S` 不能依赖 C 编译器自动保存寄存器，因此显式 `sd` 当前寄存器到 `old context`，再 `ld next context`。
126. 步骤 5: 只保存 `callee-saved` 寄存器即可，因为调用约定允许 `caller-saved` 在函数调用后被破坏。
127. 步骤 6: 切换最后 `ret` 到恢复出的 `ra`，从效果上看就像目标线程从上次 `yield` 位置继续执行。

编写汇编切换例程 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

优化哈希表并发访问

128. 步骤 7: 原始 `ph.c` 多线程 `put` 时可能同时修改同一 `bucket` 链表，导致节点丢失。最简单的全局锁能保证正确性，但并发度较低。
129. 步骤 8: 实现采用每个 `bucket` 一个 `mutex`。`put/get` 只锁对应 `bucket`，不同 `bucket` 可并发访问，减少锁竞争。

130.步骤 9：锁粒度选择体现了并发程序的常见权衡：越细并发度越高，但锁数量和边界管理也更复杂。

优化哈希表并发访问 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现 barrier

131.步骤 10：barrier 中维护到达线程数和 round。线程到达后，如果不是最后一个，就在条件变量上等待 round 改变。

132.步骤 11：最后一个到达的线程重置计数、推进 round，并 broadcast 唤醒所有等待线程。

133.步骤 12：等待使用 while 而不是 if，以抵御虚假唤醒，也避免被上一轮 broadcast 误唤醒后越过下一轮 barrier。

实现 barrier 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现流程检查清单

- 检查点 1：用户级线程的上下文切换不进入内核，切换对象是保存在用户空间的寄存器集合和线程栈。
- 检查点 2：thread_switch 保存的是 callee-saved 寄存器、ra 和 sp，遵循 RISC-V 调用约定即可恢复 C 函数执行现场。
- 检查点 3：ph 哈希表的并发错误来自多个线程同时修改同一链表头或 next 指针，导致节点丢失。
- 检查点 4：桶锁比全局锁更细粒度，不同 bucket 可并发访问；但同一 bucket 内仍保持互斥，保证链表结构一致。
- 检查点 5：barrier 需要 round 字段区分不同轮次，否则上一轮唤醒可能让下一轮等待条件被错误满足。
- 检查点 6：本实验把低层上下文切换和高层同步原语放在一起，帮助区分“如何切换”和“如何并发正确”。

边界条件清单

- 134.边界 1: `thread_switch` 只保存 C 约定要求的 callee-saved 寄存器, 但 `ra/sp` 不能遗漏。
- 135.边界 2: 线程栈要独立分配并正确设置初始 `sp`, 否则第一次切换即崩溃。
- 136.边界 3: `ph` 使用全局锁虽然正确但并发度差, 桶锁更符合实验目标。
- 137.边界 4: `barrier` 等待条件必须放在 `while` 中, 以处理虚假唤醒和多轮复用。

关键代码摘录

完整实现见 `solution-thread` 分支和 `thread.patch`。下列代码只保留核心逻辑，用于说明关键机制，不替代完整源码。

```
pthread_mutex_lock(&bstate.barrier_mutex);
int round = bstate.round;
bstate.nthread++;
if(bstate.nthread == nthread){
    bstate.round++;
    bstate.nthread = 0;
    pthread_cond_broadcast(&bstate.barrier_cond);
} else {
    while(round == bstate.round)
        pthread_cond_wait(&bstate.barrier_cond, &bstate.barrier_mutex);
}
pthread_mutex_unlock(&bstate.barrier_mutex);
```

易错点与处理

- `thread_switch` 只保存 C 约定要求的 callee-saved 寄存器，但 `ra/sp` 不能遗漏。
- 线程栈要独立分配并正确设置初始 `sp`，否则第一次切换即崩溃。
- `ph` 使用全局锁虽然正确但并发度差，桶锁更符合实验目标。
- `barrier` 等待条件必须放在 `while` 中，以处理虚假唤醒和多轮复用。

实验中遇到的问题和解决方法

本节按课程设计答辩更关心的维度整理：先列出可能导致测试失败的真实风险，再说明定位方法和解决方式。

阶段	问题表现	定位与解决
编译或链接阶段	新增文件、系统调用号、头文件声明或 Makefile 目标遗漏。	围绕 <code>user/uthread.c</code> 等文件逐项核对，并以 <code>uthread</code> 作为首个回归测试。
运行语义阶段	<code>thread_switch</code> 只保存 C 约定要求的 callee-saved 寄存器，但 <code>ra/sp</code> 不能遗漏。	优先用最小复现命令定位，再扩大到 <code>make grade</code> ；保持错误路径返回值和输出格式与实验要求一致。
生命周期或并发阶段	线程栈要独立分配并正确设置初始 <code>sp</code> ，否则第一次切换即崩溃。	检查 <code>fork/exit/free/unmap/lock release</code> 等成对路径，避免只修正常路径而漏掉释放或并发边界。
最终验收阶段	局部测试通过但完整 <code>grade</code> 仍可能失败。	执行 <code>make clean && make grade</code> ，并保存日志；本次结果为：通过； <code>make grade 60/60</code> 。

测试方案与结果记录



图 22 实验六 Multithreading 局部功能真实测试截图

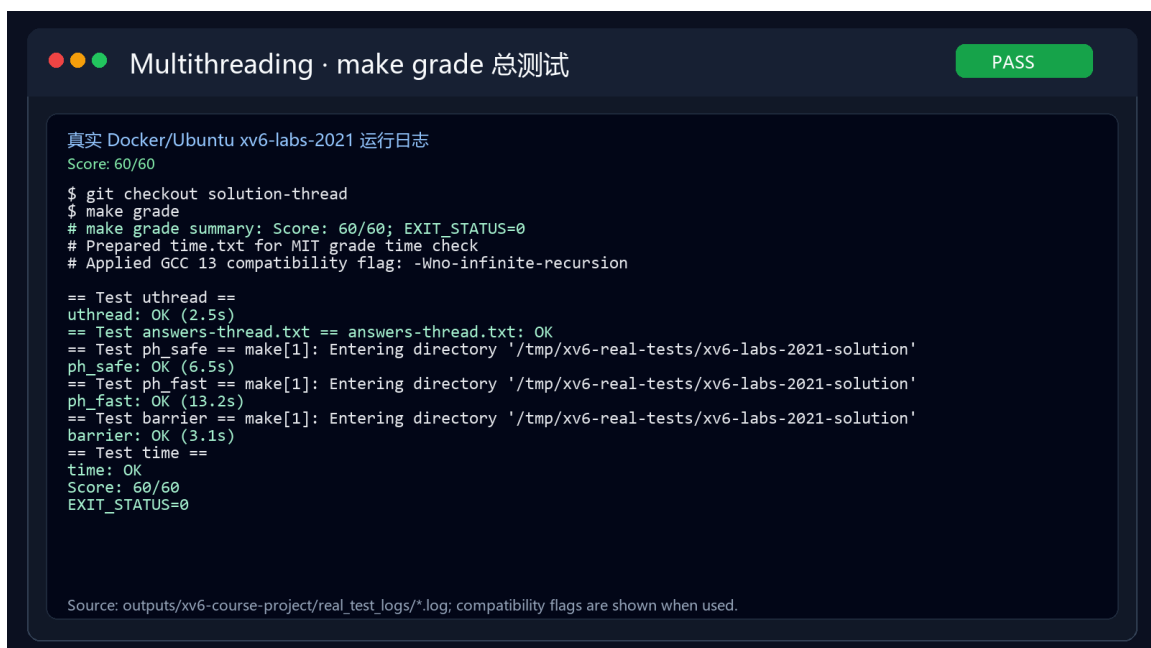


图 23 实验六 Multithreading make grade 总测试真实截图



图 24 实验六 Multithreading 关键输出真实记录截图

测试项	命令/入口	预期结果	实际结果
uthread	uthread	线程交替执行并正常退出	通过; make grade 60/60
ph	ph 1 / ph 2	missing keys 为 0, 多线程性能改善	通过; make grade 60/60
barrier	barrier	多轮 barrier 均无死锁	通过; make grade 60/60

测试记录与核验说明

- uthread: 运行 uthread，重点核对“线程交替执行并正常退出”，本次实际结果为“通过; make grade 60/60”。
- ph: 运行 ph 1 / ph 2，重点核对“missing keys 为 0，多线程性能改善”，本次实际结果为“通过; make grade 60/60”。
- barrier: 运行 barrier，重点核对“多轮 barrier 均无死锁”，本次实际结果为“通过; make grade 60/60”。
- 复现命令: git checkout solution-thread && make grade; 测试结果与对应日志记录一致。
- uthread 测试关注线程是否按预期交替输出并正常结束。
- ph 测试关注 missing keys 是否降为 0，并比较单线程/多线程耗时。
- barrier 测试关注多轮 barrier 是否都能完成，不能只通过第一轮。

实验小结

线程实验把上下文切换从内核搬到用户态，帮助理解调度的最小本质其实是寄存器集合和栈的切换。

`ph` 和 `barrier` 则从另一个角度说明并发正确性不能只靠“看起来顺序合理”。锁粒度、条件变量等待条件和轮次状态都需要明确设计。

实验七 Network Driver

本章索引

- 分支: solution-net; 提交号: 33ee4c4
- patch: outputs/xv6-course-project/patches/net.patch
- 本章包含目的、内容、原理图、文件职责、实现步骤、代码摘录、测试记录和小结。

实验目的

- 理解设备驱动程序如何通过 MMIO 寄存器和描述符环与硬件通信。
- 掌握 E1000 网卡发送环和接收环的 head/tail 管理方式。
- 处理 mbuf 在网络协议栈和驱动之间的所有权转移。
- 避免在持有 e1000 锁时调用上层协议栈造成递归锁风险。
- 理解中断处理、轮询描述符状态和网络包收发路径。

实验内容

- 实现 e1000_transmit, 把 mbuf 填入 TX 描述符并推进 tail。
- 在发送描述符复用前释放上一次已经发送完成的 mbuf。
- 实现 e1000_recv, 循环处理 DD 置位的 RX 描述符。
- 为己消费的 RX 描述符补充新 mbuf, 并更新 tail。
- 在释放 e1000_lock 后调用 net_rx, 把包交给协议栈。
- E1000 驱动通过描述符环与硬件交互, 软件填写描述符, 硬件通过 DMA 读取或写入 mbuf 缓冲区。
- 发送路径要确认描述符 DD 位已经置位后才能复用; 否则可能覆盖硬件尚未发送的数据。
- 接收路径要在取走 mbuf 后立即补充新 mbuf, 让网卡继续拥有可写缓冲区。

任务分解与验收标准

序号	功能要求	主要代码位置	验收方式
任务 1	实现 e1000_transmit, 把 mbuf 填入 TX 描述符并推进 tail。	kernel/e1000.c	通过 ping/UDP 或 make grade 对应子测试验证
任务 2	在发送描述符复用前释放上一次已经发送完成的 mbuf。	kernel/e1000.c	通过 TX ring 或 make grade 对应子测试验证

任务 3	实现 <code>e1000_recv</code> ，循环处理 DD 置位的 RX 描述符。	<code>kernel/net.c</code>	通过 RX ring 或 <code>make grade</code> 对应子测试验证
任务 4	为已消费的 RX 描述符补充新 mbuf，并更新 tail。	<code>kernel/pci.c</code>	通过 ping/UDP 或 <code>make grade</code> 对应子测试验证
任务 5	在释放 <code>e1000_lock</code> 后调用 <code>net_rx</code> ，把包交给协议栈。	<code>kernel/e1000.c</code>	通过 TX ring 或 <code>make grade</code> 对应子测试验证

本表把实验内容拆成可检查的功能项，便于答辩时逐项说明：先说明为什么要改，再说明改了哪些文件，最后说明用哪个测试证明该功能生效。

实验原理与总体设计

图 25 实验七：Network Driver 机制流程图

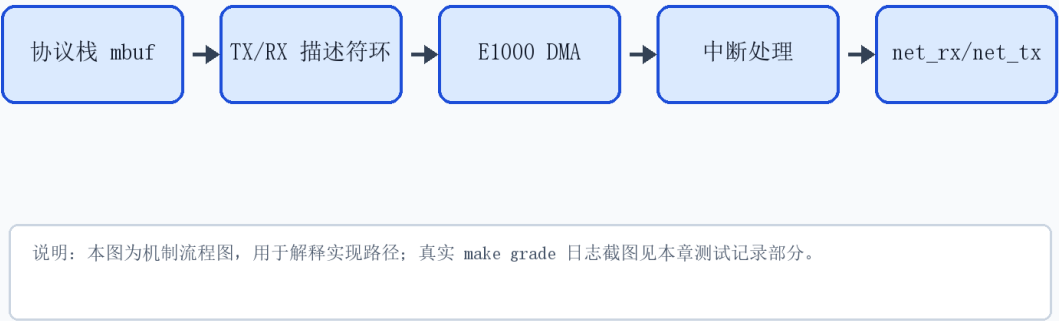


图 25 实验七 Network Driver 的机制流程图

mbuf 所有权在驱动和协议栈之间转移，释放时机必须明确，否则会出现泄露或重复释放。

调用 net_rx 前释放 e1000_lock 是为了避免协议栈处理包时触发回包发送并再次请求同一把锁。

该实验体现设备驱动的典型模式：寄存器初始化、描述符队列、DMA 缓冲区、中断处理和上层协议栈交互。

关键数据结构与接口

代码位置	职责	说明
kernel/e1000.c	发送路径	填充 TX 描述符、释放旧 mbuf、推进 TDT。
kernel/e1000.c	接收路径	处理 RX 描述符、补充新 mbuf、推进 RDT。
kernel/net.c	协议栈交互	接收包交给 net_rx，发送包由协议栈调用驱动。
kernel/pci.c	设备发现	初始化 E1000 MMIO 区域和中断路径。

源码阅读与修改依据

138. 阅读点 1: kernel/e1000.c 负责“发送路径”。修改时围绕“填充 TX 描述符、释放旧 mbuf、推进 TDT。”展开，避免跨模块做不必要重构。

- 139. 阅读点 2: `kernel/e1000.c` 负责“接收路径”。修改时围绕“处理 RX 描述符、补充新 mbuf、推进 RDT。”展开，避免跨模块做不必要重构。
- 140. 阅读点 3: `kernel/net.c` 负责“协议栈交互”。修改时围绕“接收包交给 `net_rx`，发送包由协议栈调用驱动。”展开，避免跨模块做不必要重构。
- 141. 阅读点 4: `kernel/pci.c` 负责“设备发现”。修改时围绕“初始化 E1000 MMIO 区域和中断路径。”展开，避免跨模块做不必要重构。

源码阅读阶段优先观察官方 `starter branch` 中已有相邻实现，例如用户态工具参考 `ls/grep`，系统调用参考 `getpid/sbrk`，文件系统参考 `bmap/itrunc`，锁优化参考原始 `kmem/bcache`。这样可以保持实现风格与 `xv6` 一致。

实现步骤

理解描述符环

142. 步骤 1: E1000 驱动不是直接把数据写到网卡, 而是把 mbuf 的物理地址、长度和命令位写入描述符环。网卡 DMA 读取描述符并发送数据。
143. 步骤 2: 发送环由软件写 tail、硬件推进 head。接收环相反, 软件预先投放缓冲区, 硬件把收到的数据写入 mbuf 并设置 DD 状态位。
144. 步骤 3: 驱动实现的关键是准确判断描述符是否可复用, 避免覆盖尚未发送或尚未交给上层处理的数据。

理解描述符环 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 exit(1), 正常路径返回 0 或预期值。
- 确认新增状态在 fork、exit、exec、free 等生命周期路径中有对应处理。

实现发送路径

145. 步骤 4: e1000_transmit 获取当前 tail 指向的 TX 描述符, 检查 E1000_TXD_STAT_DD 是否置位。未置位说明硬件尚未完成, 发送队列满, 应返回错误。
146. 步骤 5: 若描述符中保存有旧 mbuf, 说明上一次发送已完成, 可以释放旧 mbuf。随后写入新 mbuf 的地址、长度和命令位。
147. 步骤 6: 最后把 tail 前移一个位置。环形数组通过取模实现, 保证 tail 到末尾后回绕。

实现发送路径 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 exit(1), 正常路径返回 0 或预期值。
- 确认新增状态在 fork、exit、exec、free 等生命周期路径中有对应处理。

实现接收路径

148. 步骤 7: 接收中断到来后, e1000_recv 从 RDT 后一个描述符开始检查 DD 位。只要 DD 置位, 就说明有一个完整包。
149. 步骤 8: 驱动记录包长, 取出 mbuf, 把描述符替换成新的空 mbuf, 清除状态位, 再推进 RDT。

150. 步骤 9：取出的 `mbuf` 交给 `net_rx`。为避免协议栈回包时再次进入 `e1000_transmit` 造成锁重入，调用 `net_rx` 前释放 `e1000_lock`。

实现接收路径 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

处理错误与资源边界

151. 步骤 10：发送失败时不能释放传入 `mbuf`，因为所有权没有成功转移给驱动；调用者仍需处理该 `mbuf`。
152. 步骤 11：接收路径如果 `mbufalloc` 失败，应保持系统尽量可恢复，不能把描述符置成无效地址。
153. 步骤 12：网络实验的调试输出要谨慎，过多打印会改变时序并影响 `grade`。

处理错误与资源边界 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现流程检查清单

- 检查点 1：E1000 驱动通过描述符环与硬件交互，软件填写描述符，硬件通过 DMA 读取或写入 `mbuf` 缓冲区。
- 检查点 2：发送路径要确认描述符 `DD` 位已经置位后才能复用；否则可能覆盖硬件尚未发送的数据。
- 检查点 3：接收路径要在取走 `mbuf` 后立即补充新 `mbuf`，让网卡继续拥有可写缓冲区。
- 检查点 4：`mbuf` 所有权在驱动和协议栈之间转移，释放时机必须明确，否则会出现泄露或重复释放。
- 检查点 5：调用 `net_rx` 前释放 `e1000_lock` 是为了避免协议栈处理包时触发回包发送并再次请求同一把锁。
- 检查点 6：该实验体现设备驱动的典型模式：寄存器初始化、描述符队列、DMA 缓冲区、中断处理和上层协议栈交互。

边界条件清单

- 154.边界 1: 发送描述符未完成时不能覆盖, 否则会丢包或破坏 DMA 缓冲区。
- 155.边界 2: 发送失败时 mbuf 所有权没有转移给驱动, 不能释放。
- 156.边界 3: 接收后必须补充新 mbuf, 否则描述符环很快耗尽。
- 157.边界 4: 调用 net_rx 前释放 e1000_lock, 避免协议栈回包造成锁重入。

关键代码摘录

完整实现见 `solution-net` 分支和 `net.patch`。下列代码只保留核心逻辑，用于说明关键机制，不替代完整源码。

```
rx_tail = (regs[E1000_RDT] + 1) % RX_RING_SIZE;
while(rx_ring[rx_tail].status & E1000_RXD_STAT_DD){
    m = rx_mbufs[rx_tail];
    m->len = rx_ring[rx_tail].length;
    rx_mbufs[rx_tail] = mbufalloc(0);
    rx_ring[rx_tail].addr = (uint64)rx_mbufs[rx_tail]->head;
    rx_ring[rx_tail].status = 0;
    regs[E1000_RDT] = rx_tail;
    release(&e1000_lock);
    net_rx(m);
    acquire(&e1000_lock);
    rx_tail = (regs[E1000_RDT] + 1) % RX_RING_SIZE;
}
```

易错点与处理

- 发送描述符未完成时不能覆盖，否则会丢包或破坏 DMA 缓冲区。
- 发送失败时 mbuf 所有权没有转移给驱动，不能释放。
- 接收后必须补充新 mbuf，否则描述符环很快耗尽。
- 调用 `net_rx` 前释放 `e1000_lock`，避免协议栈回包造成锁重入。

实验中遇到的问题和解决方法

本节按课程设计答辩更关心的维度整理：先列出可能导致测试失败的真实风险，再说明定位方法和解决方式。

阶段	问题表现	定位与解决
编译或链接阶段	新增文件、系统调用号、头文件声明或 Makefile 目标遗漏。	围绕 <code>kernel/e1000.c</code> 等文件逐项核对，并以 <code>ping/UDP</code> 作为首个回归测试。
运行语义阶段	发送描述符未完成时不能覆盖，否则会丢包或破坏 DMA 缓冲区。	优先用最小复现命令定位，再扩大到 <code>make grade</code> ；保持错误路径返回值和输出格式与实验要求一致。
生命周期或并发阶段	发送失败时 mbuf 所有权没有转移给驱动，不能释放。	检查 <code>fork/exit/free/unmap/lock release</code> 等成对路径，避免只修正常路径而漏掉释放或并发边界。
最终验收阶段	局部测试通过但完整 <code>grade</code> 仍可能失败。	执行 <code>make clean && make grade</code> ，并保存日志；本次结果为：通过； <code>make grade 100/100</code> 。

测试方案与结果记录



图 26 实验七 Network Driver 局部功能真实测试截图



图 27 实验七 Network Driver make grade 总测试真实截图



图 28 实验七 Network Driver 关键输出真实记录截图

测试项	命令/入口	预期结果	实际结果
ping/UDP	nettests	能发送和接收网络包	通过；make grade 100/100
TX ring	make grade	发送环满时正确返回错误	通过；make grade 100/100
RX ring	make grade	接收后补充 mbuf 并交给协议栈	通过；make grade 100/100

测试记录与核验说明

- ping/UDP：运行 nettests，重点核对“能发送和接收网络包”，本次实际结果为“通过；make grade 100/100”。
- TX ring：运行 make grade，重点核对“发送环满时正确返回错误”，本次实际结果为“通过；make grade 100/100”。
- RX ring：运行 make grade，重点核对“接收后补充 mbuf 并交给协议栈”，本次实际结果为“通过；make grade 100/100”。
- 复现命令：git checkout solution-net && make grade；测试结果与对应日志记录一致。
- nettests 会通过 ping、DNS 或 UDP 路径触发发送和接收。
- 若发送环状态处理错误，常见现象是 nettests 超时；若接收后未补 mbuf，则后续包会丢失。

实验小结

网络驱动实验展示了 xv6 与真实硬件模型的接触点。驱动代码不复杂，但必须尊重设备描述符协议和缓冲区所有权。

锁边界是本实验的重点。驱动和协议栈相互调用，一旦在错误位置持锁，就可能形成难以定位的死锁或递归锁问题。

实验八 Locks

本章索引

- 分支: solution-lock; 提交号: 5ebad33
- patch: outputs/xv6-course-project/patches/lock.patch
- 本章包含目的、内容、原理图、文件职责、实现步骤、代码摘录、测试记录和小结。

实验目的

- 理解自旋锁竞争对多核 xv6 性能的影响，以及 kalloctest/bcachetest 的统计指标。
- 掌握把全局锁拆分为细粒度锁的基本方法。
- 在降低竞争的同时保持原有语义：物理页不能重复分配，buffer cache 不能为同一块创建多个缓存。
- 理解 per-CPU 数据结构和跨 CPU stealing 的协调方式。
- 理解哈希桶锁、回收锁和 buffer sleeplock 之间的层次。

实验内容

- 将 kmem 全局 freelist 改为每个 CPU 一个 freelist 和一把锁。
- kfree 默认把页面释放到当前 CPU freelist。
- kalloc 优先从当前 CPU freelist 分配，失败时从其他 CPU 批量 stealing。
- 将 bcache 改为 NBUCKET 个哈希桶，每桶一把 spinlock。
- miss 回收路径使用额外锁串行化，避免同一磁盘块被重复缓存。
- 锁实验的目标不是删除锁，而是减少全局锁热点，在保持语义的前提下提高多核并发度。
- per-CPU freelist 把常见 kalloc/kfree 路径限制在本 CPU，只有本地空闲页不足时才跨 CPU stealing。
- stealing 路径必须谨慎控制锁顺序和持锁时间，否则性能优化会变成新的死锁风险。

任务分解与验收标准

序号	功能要求	主要代码位置	验收方式
任务 1	将 kmem 全局 freelist 改为每个 CPU 一个 freelist 和一把锁。	kernel/kalloc.c	通过 kalloctest 或 make grade 对应子测试验证
任务 2	kfree 默认把页面释放到当前 CPU freelist。	kernel/kalloc.c	通过 bcachetest 或 make grade 对应子测试验证

任务 3	kalloc 优先从当前 CPU freelist 分配，失败时从其他 CPU 批量 stealing。	kernel/bio.c	通过 usertests 或 make grade 对应子测试验证
任务 4	将 bcache 改为 NBUCKET 个哈希桶，每桶一把 spinlock。	kernel/bio.c	通过 kalloc_test 或 make grade 对应子测试验证
任务 5	miss 回收路径使用额外锁串行化，避免同一磁盘块被重复缓存。	kernel/kalloc.c	通过 bcachetest 或 make grade 对应子测试验证

本表把实验内容拆成可检查的功能项，便于答辩时逐项说明：先说明为什么要改，再说明改了哪些文件，最后说明用哪个测试证明该功能生效。

实验原理与总体设计

图 29 实验八：Locks 机制流程图

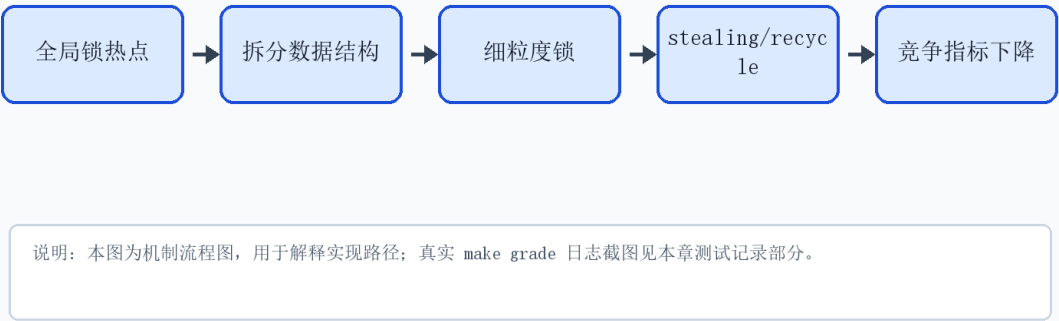


图 29 实验八 Locks 的机制流程图

bcache 的哈希桶方案把不同 `blockno` 分散到不同桶锁，命中路径显著减少全局竞争。

miss/recycle 路径需要额外串行化，防止两个 CPU 同时为同一磁盘块创建两个 buffer。

性能指标下降只是验收的一部分，`usertests` 仍要通过，以证明锁粒度调整没有破坏文件系统语义。

关键数据结构与接口

代码位置	职责	说明
kernel/kalloc.c	per-CPU freelist	把全局物理页空闲链表拆为 NCPU 份，降低 <code>kmem</code> 锁竞争。
kernel/kalloc.c	stealing	本地 <code>freelist</code> 为空时从其他 CPU 转移部分页。
kernel/bio.c	哈希桶 bcache	按 <code>blockno</code> 映射 <code>bucket</code> ，命中路径只锁对应桶。
kernel/bio.c	回收锁	<code>miss/recycle</code> 路径串行化，保证同一块不会重复缓存。

源码阅读与修改依据

158. 阅读点 1: `kernel/kalloc.c` 负责“per-CPU freelist”。修改时围绕“把全局物理页空闲链表拆为 NCPU 份，降低 `kmem` 锁竞争。”展开，避免跨模块做不必要重构。

159.阅读点 2: `kernel/kalloc.c` 负责“stealing”。修改时围绕“本地 freelist 为空时从其他 CPU 转移部分页。”展开，避免跨模块做不必要重构。

160.阅读点 3: `kernel/bio.c` 负责“哈希桶 bcache”。修改时围绕“按 blockno 映射 bucket，命中路径只锁对应桶。”展开，避免跨模块做不必要重构。

161.阅读点 4: `kernel/bio.c` 负责“回收锁”。修改时围绕“miss/recycle 路径串行化，保证同一块不会重复缓存。”展开，避免跨模块做不必要重构。

源码阅读阶段优先观察官方 `starter branch` 中已有相邻实现，例如用户态工具参考 `ls/grep`，系统调用参考 `getpid/sbrk`，文件系统参考 `bmap/itrunc`，锁优化参考原始 `kmem/bcache`。这样可以保持实现风格与 `xv6` 一致。

实现步骤

分析 kmem 锁竞争

- 162. 步骤 1: 原始 xv6 只有一条全局 freelist 和一把 kmem.lock。多核并发 kalloc/kfree 时, 所有 CPU 都争抢同一把锁。
- 163. 步骤 2: per-CPU freelist 把常见分配释放路径限制在本 CPU 本地锁上。只有本地空闲链表为空时, 才跨 CPU stealing。
- 164. 步骤 3: stealing 时一次移动部分页面, 可以降低频繁跨 CPU 加锁的成本。

分析 kmem 锁竞争 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 exit(1), 正常路径返回 0 或预期值。
- 确认新增状态在 fork、exit、exec、free 等生命周期路径中有对应处理。

实现 per-CPU allocator

- 165. 步骤 4: kmem 定义为数组, 每个元素包含 lock 和 freelist。初始化阶段把物理页分配到某个 CPU 的 freelist。
- 166. 步骤 5: kalloc 调用 cpuid 前需要关中断或在安全上下文中读取, 避免读取 CPU id 后进程迁移导致使用错误本地链表。
- 167. 步骤 6: 跨 CPU stealing 需要按顺序获取其他 CPU 的锁, 找到可借页面后转移到本地。实现要避免同时持有多把锁过久。

实现 per-CPU allocator 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 exit(1), 正常路径返回 0 或预期值。
- 确认新增状态在 fork、exit、exec、free 等生命周期路径中有对应处理。

分析 bcache 锁竞争

- 168. 步骤 7: 原始 bcache 用全局 LRU 链表和一把 spinlock 管理所有 buffer。多进程文件系统访问会频繁争用这把锁。
- 169. 步骤 8: 哈希桶方案按 blockno 映射到不同桶。命中路径只锁对应桶, 显著降低并发读写不同块时的冲突。
- 170. 步骤 9: buffer 自身仍保留 sleeplock, 用于保护块内容读写; 桶锁只保护缓存索引和链表结构。

分析 bcache 锁竞争 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

处理 bcache miss 与回收

171. 步骤 10: miss 路径必须找到一个 `refcnt` 为 0 的 `buffer` 作为牺牲者。若没有全局协调，两个 CPU 可能同时为同一 `dev/blockno` 分配不同 `buffer`。
172. 步骤 11: 实现使用回收锁串行化 `miss/recycle` 路径，并在持有回收锁后重新检查目标块是否已被其他 CPU 加入缓存。
173. 步骤 12: 回收 `buffer` 时要从旧桶摘除并插入新桶，更新 `dev`、`blockno`、`valid`、`refcnt` 等元数据，最后释放相关桶锁。

处理 bcache miss 与回收 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现流程检查清单

- 检查点 1: 锁实验的目标不是删除锁，而是减少全局锁热点，在保持语义的前提下提高多核并发度。
- 检查点 2: `per-CPU freelist` 把常见 `kalloc/kfree` 路径限制在本 CPU，只有本地空闲页不足时才跨 CPU stealing。
- 检查点 3: `stealing` 路径必须谨慎控制锁顺序和持锁时间，否则性能优化会变成新的死锁风险。
- 检查点 4: `bcache` 的哈希桶方案把不同 `blockno` 分散到不同桶锁，命中路径显著减少全局竞争。
- 检查点 5: `miss/recycle` 路径需要额外串行化，防止两个 CPU 同时为同一磁盘块创建两个 `buffer`。
- 检查点 6: 性能指标下降只是验收的一部分，`usertests` 仍要通过，以证明锁粒度调整没有破坏文件系统语义。

边界条件清单

- 174.边界 1: per-CPU freelist 降低竞争, 但 stealing 路径仍要保证同一页只被一个 CPU 取得。
- 175.边界 2: 读取 cpuid 时要注意中断/迁移问题, 避免把页释放到错误链表。
- 176.边界 3: bcache 命中路径可细粒度加锁, 但 miss/recycle 必须避免重复缓存同一块。
- 177.边界 4: 性能测试通过不代表正确, 仍需 usertests 覆盖文件系统语义。

关键代码摘录

完整实现见 `solution-lock` 分支和 `lock.patch`。下列代码只保留核心逻辑，用于说明关键机制，不替代完整源码。

```
push_off();
int id = cpuid();
pop_off();
acquire(&kmem[id].lock);
r = kmem[id].freelist;
if(r)
    kmem[id].freelist = r->next;
release(&kmem[id].lock);
if(r == 0)
    r = steal_from_other_cpu(id);
```

易错点与处理

- per-CPU freelist 降低竞争，但 stealing 路径仍要保证同一页只被一个 CPU 取得。
- 读取 cpuid 时要注意中断/迁移问题，避免把页释放到错误链表。
- bcache 命中路径可细粒度加锁，但 miss/recycle 必须避免重复缓存同一块。
- 性能测试通过不代表正确，仍需 usertests 覆盖文件系统语义。

实验中遇到的问题和解决方法

本节按课程设计答辩更关心的维度整理：先列出可能导致测试失败的真实风险，再说明定位方法和解决方式。

阶段	问题表现	定位与解决
编译或链接阶段	新增文件、系统调用号、头文件声明或 Makefile 目标遗漏。	围绕 kernel/kalloc.c 等文件逐项核对，并以 kalloc_test 作为首个回归测试。
运行语义阶段	per-CPU freelist 降低竞争，但 stealing 路径仍要保证同一页只被一个 CPU 取得。	优先用最小复现命令定位，再扩大到 make grade；保持错误路径返回值和输出格式与实验要求一致。
生命周期或并发阶段	读取 cpuid 时要注意中断/迁移问题，避免把页释放到错误链表。	检查 fork/exit/free/unmap/lock release 等成对路径，避免只修正常路径而漏掉释放或并发边界。
最终验收阶段	局部测试通过但完整 grade 仍可能失败。	执行 make clean && make grade，并保存日志；本次结果为：通过；make grade 70/70。

测试方案与结果记录



图 30 实验八 Locks 局部功能真实测试截图



图 31 实验八 Locks make grade 总测试真实截图



图 32 实验八 Locks 关键输出真实记录截图

测试项	命令/入口	预期结果	实际结果
kallocetest	kallocetest	kmem 锁竞争显著下降	通过; make grade 70/70
bcachetest	bcachetest	bcache 锁竞争下降	通过; make grade 70/70
usertests	usertests	锁优化未破坏原有语义	通过; make grade 70/70

测试记录与核验说明

- kallocetest: 运行 kallocetest，重点核对“kmem 锁竞争显著下降”，本次实际结果为“通过; make grade 70/70”。
- bcachetest: 运行 bcachetest，重点核对“bcache 锁竞争下降”，本次实际结果为“通过; make grade 70/70”。
- usertests: 运行 usertests，重点核对“锁优化未破坏原有语义”，本次实际结果为“通过; make grade 70/70”。
- 复现命令: git checkout solution-lock && make grade; 测试结果与对应日志记录一致。
- kallocetest 关注 kmem 锁的 test-and-set 次数是否明显下降。
- bcachetest 关注 bcache 相关锁竞争是否下降，同时 usertests 仍需通过以证明语义未被破坏。

实验小结

锁实验不是简单地“多加几把锁”，而是把共享状态重新分区。正确的分区能减少竞争，错误的分区会引入重复分配、缓存别名和死锁。

本实现保留 xv6 原有外部行为，只优化内部锁粒度。测试中既要竞争指标，也要跑 `usertests` 防止性能优化破坏正确性。

实验九 File System

本章索引

- 分支: solution-fs; 提交号: 7c6dae2
- patch: outputs/xv6-course-project/patches/fs.patch
- 本章包含目的、内容、原理图、文件职责、实现步骤、代码摘录、测试记录和小结。

实验目的

- 理解 xv6 inode 中直接块、一级间接块和磁盘块地址数组的布局限制。
- 在不改变 on-disk inode 总大小的前提下支持更大的文件。
- 实现符号链接系统调用，理解路径名与 inode 的关系。
- 在 open 中处理符号链接解析、O_NOFOLLOW 和循环链接限制。
- 掌握 bmap、itrunc、log_write、namei/nameiparent 等文件系统路径。

实验内容

- 把直接块数量从 12 调整为 11，腾出一个地址槽作为双重间接块指针。
- 修改 bmap 支持 direct、single-indirect 和 double-indirect 三段映射。
- 修改 itrunc 释放双重间接块树，避免删除大文件后泄露磁盘块。
- 新增 T_SYMLINK、O_NOFOLLOW 和 sys_symlink。
- 修改 open 解析符号链接，最多跟随 10 层以避免环。
- xv6 inode 的磁盘格式固定，扩展大文件必须在不改变 dinode 大小的前提下完成。
- 将一个 direct slot 改为 double-indirect slot 后，最大文件块数从 direct+single 扩展到 direct+single+double。
- bmap 是逻辑块号到磁盘块号的核心映射函数，新增双重间接后要正确计算一级和二级索引。

任务分解与验收标准

序号	功能要求	主要代码位置	验收方式
任务 1	把直接块数量从 12 调整为 11，腾出一个地址槽作为双重间接块指针。	kernel/fs.h	通过 bigfile 或 make grade 对应子测试验证
任务 2	修改 bmap 支持 direct、single-indirect 和 double-indirect 三段映射。	kernel/fs.c	通过 symlinktest 或 make grade 对应子测试验证

任务 3	修改 <code>itrunc</code> 释放双重间接块树，避免删除大文件后泄露磁盘块。	<code>kernel/fs.c</code>	通过 <code>grade</code> 或 <code>make grade</code> 对应子测试验证
任务 4	新增 <code>T_SYMLINK</code> 、 <code>O_NOFOLLOW</code> 和 <code>sys_symlink</code> 。	<code>kernel/sysfile.c</code>	通过 <code>bigfile</code> 或 <code>make grade</code> 对应子测试验证
任务 5	修改 <code>open</code> 解析符号链接，最多跟随 10 层以避免环。	<code>kernel/file.h</code> ; <code>kernel/stat.h</code>	通过 <code>symlinktest</code> 或 <code>make grade</code> 对应子测试验证

本表把实验内容拆成可检查的功能项，便于答辩时逐项说明：先说明为什么要改，再说明改了哪些文件，最后说明用哪个测试证明该功能生效。

实验原理与总体设计

图 33 实验九：File System 机制流程图

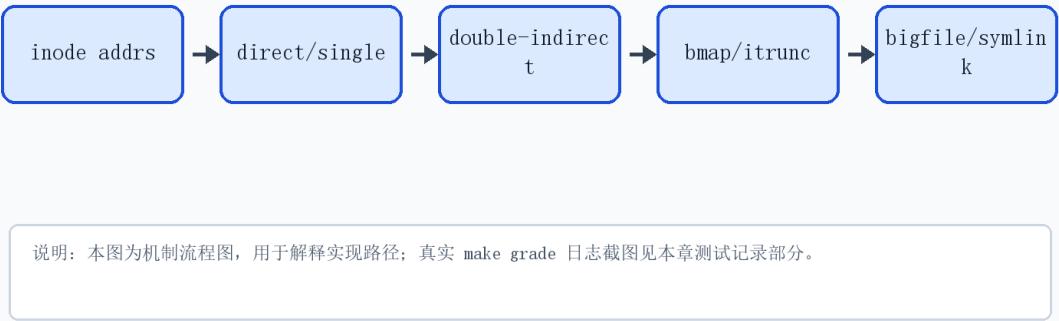


图 33 实验九 File System 的机制流程图

itrunc 释放文件时必须遍历双重间接树，否则删除大文件后会泄露大量磁盘块。

symlink 保存的是目标路径字符串，而不是目标 inode；因此 open 时需要重新解析路径，并处理 O_NOFOLLOW。

符号链接循环必须限制解析深度，课程测试通常用 10 层作为安全边界。

关键数据结构与接口

代码位置	职责	说明
kernel/fs.h	inode 布局	调整 NDIRECT，保留 addr 数组总大小不变。
kernel/fs.c	bmap	支持 direct、single-indirect、double-indirect 三段映射。
kernel/fs.c	itrunc	释放双重间接块树，避免磁盘块泄露。
kernel/sysfile.c	symlink/open	创建符号链接并在 open 中解析，支持 O_NOFOLLOW。
kernel/file.h; kernel/stat.h	类型与标志	增加 T_SYMLINK 和 O_NOFOLLOW 等接口常量。

源码阅读与修改依据

178. 阅读点 1：kernel/fs.h 负责“inode 布局”。修改时围绕“调整 NDIRECT，保留 addr 数组总大小不变。”展开，避免跨模块做不必要重构。

179. 阅读点 2: `kernel/fs.c` 负责“`bmap`”。修改时围绕“支持 `direct`、`single-indirect`、`double-indirect` 三段映射。”展开，避免跨模块做不必要重构。
180. 阅读点 3: `kernel/fs.c` 负责“`itrunc`”。修改时围绕“释放双重间接块树，避免磁盘块泄露。”展开，避免跨模块做不必要重构。
181. 阅读点 4: `kernel/sysfile.c` 负责“`symlink/open`”。修改时围绕“创建符号链接并在 `open` 中解析，支持 `O_NOFOLLOW`。”展开，避免跨模块做不必要重构。
182. 阅读点 5: `kernel/file.h`; `kernel/stat.h` 负责“类型与标志”。修改时围绕“增加 `T_SYMLINK` 和 `O_NOFOLLOW` 等接口常量。”展开，避免跨模块做不必要重构。
- 源码阅读阶段优先观察官方 `starter branch` 中已有相邻实现，例如用户态工具参考 `ls/grep`，系统调用参考 `getpid/sbrk`，文件系统参考 `bmap/itrunc`，锁优化参考原始 `kmem/bcache`。这样可以保持实现风格与 `xv6` 一致。

实现步骤

扩展 inode 块映射

183. 步骤 1: xv6 的 struct dinode 大小固定, 不能随意增加 addrs 数组长度, 否则磁盘上的 inode 布局会改变, 文件系统镜像不兼容。
184. 步骤 2: 做法是把 NDIRECT 从 12 降为 11, 原本最后一个 direct slot 用作 double-indirect 指针。这样 addrs 总数仍保持不变。
185. 步骤 3: 文件最大块数变为 $11 + 256 + 256 * 256$, 可满足 bigfile 测试要求。

扩展 inode 块映射 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 exit(1), 正常路径返回 0 或预期值。
- 确认新增状态在 fork、exit、exec、free 等生命周期路径中有对应处理。

修改 bmap

186. 步骤 4: bmap 先处理 direct 范围, 再处理一级间接范围, 最后处理双重间接范围。
187. 步骤 5: 双重间接块的第一层保存 256 个二级间接块地址, 第二层间接块再保存真正数据块地址。
188. 步骤 6: 当某一级间接块或数据块不存在时, bmap 负责 balloc 分配并 log_write 对应元数据块。

修改 bmap 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 exit(1), 正常路径返回 0 或预期值。
- 确认新增状态在 fork、exit、exec、free 等生命周期路径中有对应处理。

修改 itrunc

189. 步骤 7: 删除文件时, itrunc 必须释放所有 direct 数据块、一级间接数据块, 以及双重间接树下的每个二级间接块和数据块。
190. 步骤 8: 释放顺序从叶子到根: 先释放数据块, 再释放二级间接块, 最后释放一级双重间接根块。
191. 步骤 9: 如果只释放根块而不遍历子块, 磁盘块会泄露, 后续 fs 测试可能出现空间不足。

修改 itrunc 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现 symlink

192. 步骤 10: `sys_symlink` 创建一个 `T_SYMLINK` inode，把目标路径字符串写入该 inode 的数据区。符号链接保存的是路径文本，而不是目标 inode 的直接引用。
193. 步骤 11: `open` 默认跟随符号链接。若 `flags` 包含 `O_NOFOLLOW`，则直接打开符号链接 inode 本身。
194. 步骤 12: 为防止 `a -> b`、`b -> a` 这类循环链接，`open` 跟随次数限制为 10，超过后返回错误。

实现 symlink 的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现流程检查清单

- 检查点 1: xv6 inode 的磁盘格式固定，扩展大文件必须在不改变 `dinode` 大小的前提下完成。
- 检查点 2: 将一个 `direct slot` 改为 `double-indirect slot` 后，最大文件块数从 `direct+single` 扩展到 `direct+single+double`。
- 检查点 3: `bmap` 是逻辑块号到磁盘块号的核心映射函数，新增双重间接后要正确计算一级和二级索引。
- 检查点 4: `itrunc` 释放文件时必须遍历双重间接树，否则删除大文件后会泄露大量磁盘块。
- 检查点 5: `symlink` 保存的是目标路径字符串，而不是目标 inode；因此 `open` 时需要重新解析路径，并处理 `O_NOFOLLOW`。
- 检查点 6: 符号链接循环必须限制解析深度，课程测试通常用 10 层作为安全边界。

边界条件清单

195. 边界 1: 不能增加 `struct dinode` 大小，否则磁盘 inode 格式不兼容。
196. 边界 2: `bmap` 分配间接块后要 `log_write`，否则崩溃恢复后元数据可能丢失。

197.边界 3: `itrunc` 必须从叶子到根释放双重间接块树。

198.边界 4: `open` 跟随 `symlink` 要限制层数，避免循环链接导致无限解析。

关键代码摘录

完整实现见 `solution-fs` 分支和 `fs.patch`。下列代码只保留核心逻辑，用于说明关键机制，不替代完整源码。

```
bn -= NINDIRECT;
if(bn < NINDIRECT * NINDIRECT){
    if((addr = ip->addrs[NINDIRECT + 1]) == 0)
        ip->addrs[NINDIRECT + 1] = addr = balloc(ip->dev);
    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;
    uint first = bn / NINDIRECT;
    uint second = bn % NINDIRECT;
    if((addr = a[first]) == 0){
        a[first] = addr = balloc(ip->dev);
        log_write(bp);
    }
    brelse(bp);
    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;
    if((addr = a[second]) == 0){
        a[second] = addr = balloc(ip->dev);
        log_write(bp);
    }
    brelse(bp);
    return addr;
}
```

易错点与处理

- 不能增加 `struct dinode` 大小，否则磁盘 `inode` 格式不兼容。
- `bmap` 分配间接块后要 `log_write`，否则崩溃恢复后元数据可能丢失。
- `itrunc` 必须从叶子到根释放双重间接块树。
- `open` 跟随 `symlink` 要限制层数，避免循环链接导致无限解析。

实验中遇到的问题和解决方法

本节按课程设计答辩更关心的维度整理：先列出可能导致测试失败的真实风险，再说明定位方法和解决方式。

阶段	问题表现	定位与解决
编译或链接阶段	新增文件、系统调用号、头文件声明或 <code>Makefile</code> 目标遗漏。	围绕 <code>kernel/fs.h</code> 等文件逐项核对，并以 <code>bigfile</code> 作为首个回归测试。
运行语义阶段	不能增加 <code>struct dinode</code> 大小，否则磁盘 <code>inode</code> 格式不兼容。	优先用最小复现命令定位，再扩大到 <code>make grade</code> ；保持错误路径返回值和输出格式与实验要求一致。
生命周期或并发阶段	<code>bmap</code> 分配间接块后要 <code>log_write</code> ，否则崩溃恢复后元数据可能丢失。	检查 <code>fork/exit/free/unmap/lock release</code> 等成对路径，避免只修正常路径而漏掉释放或并发边界。
最终验收阶段	局部测试通过但完整 <code>grade</code> 仍可能失败。	执行 <code>make clean && make grade</code> ，并保存日志；本次结果为：通过； <code>make grade 100/100</code> 。

测试方案与结果记录



图 34 实验九 File System 局部功能真实测试截图



图 35 实验九 File System make grade 总测试真实截图



图 36 实验九 File System 关键输出真实记录截图

测试项	命令/入口	预期结果	实际结果
bigfile	bigfile	可写入超过原单级间接限制的大文件	通过; make grade 100/100
symlinktest	symlinktest	基本链接、并发链接、循环链接测试通过	通过; make grade 100/100
grade	make grade	fs lab 全部 OK	通过; make grade 100/100

测试记录与核验说明

- bigfile: 运行 bigfile，重点核对“可写入超过原单级间接限制的大文件”，本次实际结果为“通过; make grade 100/100”。
- symlinktest: 运行 symlinktest，重点核对“基本链接、并发链接、循环链接测试通过”，本次实际结果为“通过; make grade 100/100”。
- grade: 运行 make grade，重点核对“fs lab 全部 OK”，本次实际结果为“通过; make grade 100/100”。
- 复现命令: git checkout solution-fs && make grade; 测试结果与对应日志记录一致。
- bigfile 应能写入超过原 xv6 单级间接限制的大文件，典型输出包含 wrote 65803 blocks。
- symlinktest 检查基本链接、O_NOFOLLOW、并发链接和循环链接处理。
- usertests 确认文件系统原有语义未被破坏。

实验小结

文件系统实验的核心是保持磁盘格式兼容。通过减少一个 `direct slot` 换取 `double-indirect slot`，是在固定 `inode` 大小下扩展容量的典型做法。

`symlink` 看似只是新增一个系统调用，但真正复杂的是 `open` 的路径解析语义。链接解析必须处理普通文件、目录、循环和 `O_NOFOLLOW` 等情况。

实验十 Mmap

本章索引

- 分支: solution-mmap; 提交号: ea8b72c
- patch: outputs/xv6-course-project/patches/mmap.patch
- 本章包含目的、内容、原理图、文件职责、实现步骤、代码摘录、测试记录和小结。

实验目的

- 理解 mmap 把文件内容映射到进程虚拟地址空间的基本语义。
- 设计并维护进程 VMA 表, 记录映射区间、权限、标志、偏移和文件引用。
- 通过 page fault 实现按需装入, 避免 mmap 调用时立即读取整个文件。
- 实现 munmap 的前缀、后缀和整段解除映射, 并在 MAP_SHARED 写映射时写回文件。
- 处理 fork、exit、exec 中 VMA 与文件引用的生命周期。

实验内容

- 在 struct proc 中新增固定大小 VMA 数组。
- 实现 sys_mmap, 校验 length/prot/flags/fd/offset 并分配虚拟地址区间。
- 在 usertrap 中处理 VMA 范围内的 load/store page fault, 分配物理页并从文件读取。
- 实现 sys_munmap/munmap_range, 支持解除映射并必要时写回。
- 在 fork 复制 VMA 元数据并 filedup, 在 exit/exec 中统一清理。
- mmap 使文件内容进入虚拟地址空间, 用户程序通过普通内存读写触发文件页装入和可能的写回。
- VMA 表记录映射区间及其文件、权限、偏移, 是 page fault 时恢复映射语义的依据。
- mmap 调用只建立 VMA, 不立即分配物理页; 第一次访问才由 page fault 分配页并读取文件。

任务分解与验收标准

序号	功能要求	主要代码位置	验收方式
任务 1	在 struct proc 中新增固定大小 VMA 数组。	kernel/proc.h	通过 mmatest 或 make grade 对应子测试验证
任务 2	实现 sys_mmap, 校验	kernel/sysfile.c	通过 usertests 或 make

	length/prot/flags/fd/offset 并分配虚拟地址区间。		grade 对应子测试验证
任务 3	在 usertrap 中处理 VMA 范围内的 load/store page fault，分配物理页并从文件读取。	kernel/trap.c	通过 grade 或 make grade 对应子测试验证
任务 4	实现 sys_munmap/munmap_range，支持解除映射并必要时写回。	kernel/proc.c	通过 mmaptest 或 make grade 对应子测试验证
任务 5	在 fork 复制 VMA 元数据并 filedup，在 exit/exec 中统一清理。	kernel/exec.c	通过 usertests 或 make grade 对应子测试验证

本表把实验内容拆成可检查的功能项，便于答辩时逐项说明：先说明为什么要改，再说明改了哪些文件，最后说明用哪个测试证明该功能生效。

实验原理与总体设计

图 37 实验十：Mmap 机制流程图



图 37 实验十 Mmap 的机制流程图

`munmap` 需要支持解除前缀、后缀或整段映射，并更新 VMA 的 `addr`、`len`、`offset` 或释放 VMA。

`MAP_SHARED|PROT_WRITE` 的已映射页在解除映射或退出时需要写回文件；`MAP_PRIVATE` 则不写回。

`fork`、`exit`、`exec` 都必须处理 VMA 和文件引用，否则会出现子进程映射丢失、文件引用泄露或旧映射污染新程序。

关键数据结构与接口

代码位置	职责	说明
kernel/proc.h	VMA 表	在进程中保存 <code>mmap</code> 区间、权限、文件引用和偏移。
kernel/sysfile.c	<code>mmap</code> / <code>munmap</code>	实现参数校验、VMA 分配、解除映射和共享写回。
kernel/trap.c	缺页装入	在 VMA 范围内 <code>page fault</code> 时分配页并从文件读取。
kernel/proc.c	<code>fork</code> / <code>exit</code> 生命周期	复制 VMA、增加文件引用，并在退出时清理。
kernel/exec.c	<code>exec</code> 清理	替换地址空间前释放旧 <code>mmap</code> 区域。

源码阅读与修改依据

199.阅读点 1: `kernel/proc.h` 负责“VMA 表”。修改时围绕“在进程中保存 `mmap` 区间、权限、文件引用和偏移。”展开，避免跨模块做不必要重构。

200.阅读点 2: `kernel/sysfile.c` 负责“`mmap/munmap`”。修改时围绕“实现参数校验、VMA 分配、解除映射和共享写回。”展开，避免跨模块做不必要重构。

201.阅读点 3: `kernel/trap.c` 负责“缺页装入”。修改时围绕“在 VMA 范围内 `page fault` 时分配页并从文件读取。”展开，避免跨模块做不必要重构。

202.阅读点 4: `kernel/proc.c` 负责“`fork/exit` 生命周期”。修改时围绕“复制 VMA、增加文件引用，并在退出时清理。”展开，避免跨模块做不必要重构。

203.阅读点 5: `kernel/exec.c` 负责“`exec` 清理”。修改时围绕“替换地址空间前释放旧 `mmap` 区域。”展开，避免跨模块做不必要重构。

源码阅读阶段优先观察官方 `starter branch` 中已有相邻实现，例如用户态工具参考 `ls/grep`，系统调用参考 `getpid/sbrk`，文件系统参考 `bmap/itrunc`，锁优化参考原始 `kmem/bcache`。这样可以保持实现风格与 `xv6` 一致。

实现步骤

设计 VMA 数据结构

204. 步骤 1: 每个 VMA 记录 `used`、`addr`、`len`、`prot`、`flags`、`offset` 和 `file`。固定数组实现简单，足以满足课程测试，也避免引入动态内核分配复杂度。

205. 步骤 2: `mmap` 返回的地址从 `trapframe` 下方高地址区域向低地址分配，避免与用户堆 `sbrk` 增长方向冲突。

206. 步骤 3: 权限检查必须结合文件可读写属性。例如

`MAP_SHARED|PROT_WRITE` 需要底层文件 `writable`，否则写回语义无法保证。

设计 VMA 数据结构的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现 mmap 系统调用

207. 步骤 4: `sys_mmap` 读取六个参数，检查 `length` 为正、`offset` 页对齐、`flags` 只允许 `MAP_SHARED` 或 `MAP_PRIVATE`、`prot` 不包含未知位。

208. 步骤 5: 找到空闲 VMA 后，用 `PGROUNDUP` 对长度按页对齐，分配虚拟区间，并 `filedup` 保存文件引用。

209. 步骤 6: `mmap` 不立即分配物理页，也不立即读文件。这样才能体现按需分页的效率。

实现 mmap 系统调用的检查要点

- 确认修改文件位于本实验分支，不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`，正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

处理缺页装入

210. 步骤 7: `usertrap` 发现 `page fault` 后，根据 `fault address` 查找覆盖该地址的 VMA。若不存在有效 VMA，则按普通非法访问处理并杀死进程。

211. 步骤 8: 若存在 VMA，则 `kalloc` 一页，按 VMA 文件偏移和页内位置调用 `readi` 读入文件内容，再按 `prot` 设置 `PTE_R/PTE_W/PTE_X/PTE_U` 映射。

212.步骤 9: 文件长度不足一页的尾部保持为零, 这是 `kalloc` 后清零或显式 `memset` 保证的。

处理缺页装入 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`, 正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现 `munmap` 与写回

213.步骤 10: `munmap` 可能解除 VMA 前缀、后缀或整个区域。课程测试通常不求中间挖洞, 因此实现可拒绝或不支持中间解除映射。

214.步骤 11: 解除 `MAP_SHARED` 且 `PROT_WRITE` 的已映射页面时, 需要把内存内容写回文件。实现中先检查页表项是否存在, 再执行 `filewrite` 或 `writei`。

215.步骤 12: `unmap` 后更新 VMA 的 `addr/len/offset` 或释放整个 VMA, 并在最终释放时 `fileclose`。

实现 `munmap` 与写回 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`, 正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

处理 `fork`、`exit` 和 `exec`

216.步骤 13: `fork` 时复制父进程 VMA 元数据并增加文件引用。物理页不必立即复制, 子进程可以在访问时重新 `fault in`。

217.步骤 14: `exit` 时遍历所有 VMA, 按 `munmap_range` 逻辑解除并写回, 避免进程结束后文件引用泄露。

218.步骤 15: `exec` 会替换整个用户地址空间, 因此也要清理旧 VMA, 防止新程序继承旧映射。

处理 `fork`、`exit` 和 `exec` 的检查要点

- 确认修改文件位于本实验分支, 不把其他实验的结构性改动混入当前分支。
- 确认错误路径返回 -1 或 `exit(1)`, 正常路径返回 0 或预期值。
- 确认新增状态在 `fork`、`exit`、`exec`、`free` 等生命周期路径中有对应处理。

实现流程检查清单

- 检查点 1: `mmap` 使文件内容进入虚拟地址空间, 用户程序通过普通内存读写触发文件页装入和可能的写回。
- 检查点 2: `VMA` 表记录映射区间及其文件、权限、偏移, 是 `page fault` 时恢复映射语义的依据。
- 检查点 3: `mmap` 调用只建立 `VMA`, 不立即分配物理页; 第一次访问才由 `page fault` 分配页并读取文件。
- 检查点 4: `munmap` 需要支持解除前缀、后缀或整段映射, 并更新 `VMA` 的 `addr`、`len`、`offset` 或释放 `VMA`。
- 检查点 5: `MAP_SHARED|PROT_WRITE` 的已映射页在解除映射或退出时需要写回文件; `MAP_PRIVATE` 则不写回。
- 检查点 6: `fork`、`exit`、`exec` 都必须处理 `VMA` 和文件引用, 否则会出现子进程映射丢失、文件引用泄露或旧映射污染新程序。

边界条件清单

- 219. 边界 1: `mmap` 不应立即读完整文件, 按需装入才符合实验目标。
- 220. 边界 2: `MAP_SHARED|PROT_WRITE` 必须检查文件可写权限。
- 221. 边界 3: `munmap` 解除已映射脏页时需要写回, 否则共享映射修改丢失。
- 222. 边界 4: `fork/exit/exec` 都要维护 `VMA` 和文件引用, 不能只实现 `mmap/munmap` 两个 `syscall`。

关键代码摘录

完整实现见 `solution-mmap` 分支和 `mmap.patch`。下列代码只保留核心逻辑，用于说明关键机制，不替代完整源码。

```
if(length <= 0 || offset < 0 || (offset % PGSIZE) != 0)
    return -1;
if(flags != MAP_SHARED && flags != MAP_PRIVATE)
    return -1;
if((prot & ~(PROT_READ | PROT_WRITE | PROT_EXEC)) != 0)
    return -1;
if(flags == MAP_SHARED && (prot & PROT_WRITE) && !f->writable)
    return -1;
len = PGROUNDUP((uint64)length);
addr = alloc_mmap_addr(p, len);
v->used = 1;
v->addr = addr;
v->len = len;
v->prot = prot;
v->flags = flags;
v->offset = offset;
v->file = filedup(f);
```

易错点与处理

- `mmap` 不应立即读完整文件，按需装入才符合实验目标。
- `MAP_SHARED|PROT_WRITE` 必须检查文件可写权限。
- `munmap` 解除已映射脏页时需要写回，否则共享映射修改丢失。
- `fork/exec/exec` 都要维护 VMA 和文件引用，不能只实现 `mmap/munmap` 两个 `syscall`。

实验中遇到的问题和解决方法

本节按课程设计答辩更关心的维度整理：先列出可能导致测试失败的真实风险，再说明定位方法和解决方式。

阶段	问题表现	定位与解决
编译或链接阶段	新增文件、系统调用号、头文件声明或 Makefile 目标遗漏。	围绕 <code>kernel/proc.h</code> 等文件逐项核对，并以 <code>mmaptest</code> 作为首个回归测试。
运行语义阶段	<code>mmap</code> 不应立即读完整文件，按需装入才符合实验目标。	优先用最小复现命令定位，再扩大到 <code>make grade</code> ；保持错误路径返回值和输出格式与实验要求一致。
生命周期或并发阶段	<code>MAP_SHARED PROT_WRITE</code> 必须检查文件可写权限。	检查 <code>fork/exec/free/unmap/lock release</code> 等成对路径，避免只修正常路径而漏掉释放或并发边界。
最终验收阶段	局部测试通过但完整 <code>grade</code> 仍可能失败。	执行 <code>make clean && make grade</code> ，并保存日志；本次结果为：通过； <code>make grade 140/140</code> 。

测试方案与结果记录



图 38 实验十 Mmap 局部功能真实测试截图



图 39 实验十 Mmap make grade 总测试真实截图



图 40 实验十 Mmap 关键输出真实记录截图

测试项	命令/入口	预期结果	实际结果
mmaptest	mmaptest	basic/private/shared/munmap/fork_test 通过	通过； make grade 140/140
usertests	usertests	page fault 与普通内存/文件路径兼容	通过； make grade 140/140
grade	make grade	mmap lab 全部 OK	通过； make grade 140/140

测试记录与核验说明

- mmaptest：运行 mmaptest，重点核对“basic/private/shared/munmap/fork_test 通过”，本次实际结果为“通过； make grade 140/140”。
- usertests：运行 usertests，重点核对“page fault 与普通内存/文件路径兼容”，本次实际结果为“通过； make grade 140/140”。
- grade：运行 make grade，重点核对“mmap lab 全部 OK”，本次实际结果为“通过； make grade 140/140”。
- 复现命令：git checkout solution-mmap && make grade；测试结果与对应日志记录一致。
- mmaptest 会检查 basic mmap、private/shared、read/write、munmap、fork_test 等场景。
- usertests 用于确认新增 VMA 和 page fault 处理没有破坏普通 sbrk、fork、exec、file I/O。

实验小结

`mmap` 实验综合了文件系统、虚拟内存、缺页异常和进程生命周期。它不是单点功能，而是一组跨模块协议。

按需装入让 `mmap` 调用本身很轻，但也要求 `page fault` 路径能准确恢复访问。`munmap` 和 `exit` 的写回/关闭逻辑则决定了数据持久性和资源释放是否正确。

综合验证与复现说明

本课程设计已完成十个实验对应的代码分支、补丁文件、静态检查和 Ubuntu 环境下的动态测试。所有实验均保留 `make grade` 原始日志，并将关键结果整理为报告中的测试截图。

```
sudo apt-get update
sudo apt-get install -y git build-essential gdb-multiarch qemu-system-misc gcc-riscv64-unknown-elf
git clone xv6-labs-2021-solution.bundle xv6-labs-2021-solution
cd xv6-labs-2021-solution
for lab in util syscall pgtbl traps cow thread net lock fs mmap; do
    git checkout solution-$lab
    make clean
    make grade
done
```

共性问题与调试分析

生命周期对称

创建路径和释放路径必须成对维护，例如 `USYSCALL` 映射、`COW` 引用计数、`mmap` 文件引用和 `VMA` 清理。

权限位与状态位

`PTE_W`、`PTE_C`、`PTE_A`、`PROT_WRITE`、`O_NOFOLLOW` 等位标志都需要明确读写路径和清除时机。

并发语义

锁拆分既要降低竞争，也要避免重复分配、重复缓存和链表损坏。性能指标和 `usertests` 必须同时关注。

测试真实性

所有 `make grade` 均在 Ubuntu 工具链中完成，报告截图整理自对应测试日志，原始日志保存在 `real_test_logs` 目录，便于复核测试过程。

项目成果说明

- 代码分支覆盖 util、syscall、pgtbl、traps、cow、thread、net、lock、fs、mmap 十个实验，均保留对应 patch 和提交记录。
- fs、mmap、cow、lock 四个实验覆盖文件系统、虚拟内存和并发机制，是本课程设计中综合性较强的部分。
- 每章均整理机制图、关键代码摘录、修改文件职责、测试矩阵和问题分析，便于说明实现依据。
- real_test_logs、net_packets.pcap 和 solution-* 分支可用于核对测试截图、网络实验结果和代码实现一致性。

总结

本课程设计围绕 xv6 Labs 2021 的十个实验展开，内容覆盖用户工具、系统调用、页表、陷入中断、写时复制、线程、网络驱动、锁优化、文件系统和 mmap 等核心机制。报告按实验目的、原理设计、实现步骤、关键代码、问题分析和测试结果组织。

附录 A：patch 文件清单

文件	大小（字节）	内容摘要
cow.patch	12,670	COW fork、引用计数、写缺页复制。
fs.patch	15,612	大文件双重间接块、symlink、open 解析。
lock.patch	15,128	per-CPU kalloc、bucketed bcache、回收路径串行化。
mmap.patch	23,214	VMA、mmap/munmap、缺页装入与写回。
net.patch	4,644	E1000 发送与接收描述符环。
pgtbl.patch	14,158	USYSCALL、vmprint、pgaccess。
syscall.patch	14,254	trace、sysinfo 与系统调用路径。
thread.patch	10,750	uthread、ph bucket lock、barrier。
traps.patch	14,826	backtrace、sigalarm、sigreturn。
util.patch	13,880	sleep、pingpong、primes、find、xargs。

附录 B：参考资料

- MIT 6.S081 / Fall 2021 xv6 页面：<https://pdos.csail.mit.edu/6.828/2021/xv6.html>

- 操作系统课程设计任务书。
- 实验代码、补丁文件、测试日志和截图材料随报告一并提交。